



Patrones de software

Implementación de patrones v8

Docente: ELIECER MONTERO OJEDA

Ángel David Reyes Moreno

UNIDADES TECNOLÓGICAS DE SANTANDER
Ingeniería de Sistemas
Bucaramanga, 2026

PATRONES CREACIONALES

Patrón 1: Singleton

¿Qué problema resuelve? Garantiza que solo exista una única instancia de un objeto durante toda la vida de la aplicación. Útil para configuración y conexiones a bases de datos.

```
export class ConfigManager {  
  // 1. Variable estática que guarda LA única instancia  
  private static instance: ConfigManager | null = null;  
  
  // 2. Constructor PRIVADO → nadie puede hacer "new ConfigManager()"  
  private constructor() {  
    this.config = { /* Lee las variables de entorno */ }  
  }  
  
  // 3. Punto de acceso global → siempre devuelve la misma instancia  
  public static getInstance(): ConfigManager {  
    if (!ConfigManager.instance) {  
      ConfigManager.instance = new ConfigManager(); // solo se crea UNA vez  
    }  
    return ConfigManager.instance; // las demás veces, devuelve la existente  
  }  
}
```

¿Dónde se usa? El cliente de Supabase lo llama al iniciarse:

```
let supabaseInstance: SupabaseClient | null = null;  
  
export function getSupabaseClient(): SupabaseClient {  
  if (supabaseInstance) return supabaseInstance; // ya existe → devuelve la misma  
  
  const config = ConfigManager.getInstance(); // + usa el otro Singleton  
  supabaseInstance = createClient(config.get("supabaseUrl"), ...);  
  return supabaseInstance;  
}
```

Diagrama Uml

PATRÓN SINGLETON

getSupabaseClient()



Patrón 2: Factory Method

¿Qué problema resuelve? Delegar la **creación de objetos** a subclases, sin que el código cliente sepa qué clase concreta se está creando. Perfecto para múltiples tipos de dispositivos IoT.

```
// Creator abstracto define el "molde"
abstract class DeviceAdapterCreator {
  abstract createAdapter(id: string, type: DeviceType): IDeviceAdapter; // ← Factory Method
}

// Cada subclase decide QUÉ adaptador concreto crear
class SimulatedAdapterCreator extends DeviceAdapterCreator {
  createAdapter(id: string, type: DeviceType): IDeviceAdapter {
    return new SimulatedDeviceAdapter(id, type); // concreta
  }
}

// Punto de entrada público — elige el Creator correcto
export class DeviceAdapterFactory {
  static async create(params): Promise<IDeviceAdapter> {
    switch (params.protocol) {
      case "MQTT":      return new MQTTAdapterCreator().getAndConnect(...);
      case "REST":      return new RESTAdapterCreator().getAndConnect(...);
      case "SIMULATED": return new SimulatedAdapterCreator().getAndConnect(...);
    }
  }
}
```

¿Dónde se usa en la UI?

La UI solo llama a la fábrica. De momento se está instanciando un protocolo de simulación si en un futuro se implementa un protocolo en tiempo real solo hay que crear ese protocolo ejemplo: ZIGBEE, solo ZigbeeDeviceAdapter y ZigbeeAdapterCreator — el código de la UI no cambia para nada seguirá llamando a la fábrica .

```
// La página NO sabe qué adaptador concreto se crea
const adapter = await DeviceAdapterFactory.create({
  deviceId: "dev-001",
  deviceType: "SOLAR_PANEL",
  protocol: "SIMULATED", // ← aquí solo cambia el protocolo
});
const reading = await adapter.readData(); // misma interfaz siempre
```

Diagrama Uml

FACTORY METHOD — Device Adapter

Patron de disenio creacional

«interface»

IDeviceAdapter

connect() · readData() · disconnect()

Contrato de comunicacion con dispositivos

implementa

implementaciones concretas

MQTTDeviceAdapter

connect()

(fisica real)

RESTDeviceAdapter

connect()

(API HTTP)

SimulatedDeviceAdapter

connect()^{crea / usa}

(datos sinteticos)

«abstract»

DeviceAdapterCreator

createAdapter() — Factory Method

subclases deben implementar createAdapter()

extiende

creators concretos

MQTTAdapterCreator

createAdapter()

retorna instancia concreta

RESTAdapterCreator

createAdapter()

retorna instancia concreta

SimulatedAdapterCreator

createAdapter()

retorna instancia concreta

★ Cada Creator sabe cual Adapter crear — el cliente solo habla con IDeviceAdapter

Patrón 3: Abstract Factory

¿Qué problema resuelve?

En la plataforma tenemos **4 fuentes de energía** (Solar, Eólica, Batería, Red). Cada una necesita tres componentes que **deben ser coherentes entre sí**:

- Un lector de producción
- Un calculador de precios
- Un modelo de predicción

Sin el patrón, la UI tendría que saber qué clase concreta instanciar en cada caso, generando código espagueti

¿Dónde se usa?

Se define las tres faces del producto

```
interface IEnergyProducer { getCurrentReading(): Promise<EnergyReading>; }
interface IPriceEstimator { getCurrentQuote(): Promise<PriceQuote>; }
interface IForecastModel { getDailyForecast(): Promise<DailyForecast>; }
```

Se define la Abstract Factory (el contrato de la fábrica)

```
interface IEnergySourceFactory {
  createProducer(): IEnergyProducer; // + crea producto A
  createPriceEstimator(): IPriceEstimator; // + crea producto B
  createForecastModel(): IForecastModel; // + crea producto C
}
```

Se implementan las 4 fábricas concretas, una por familia

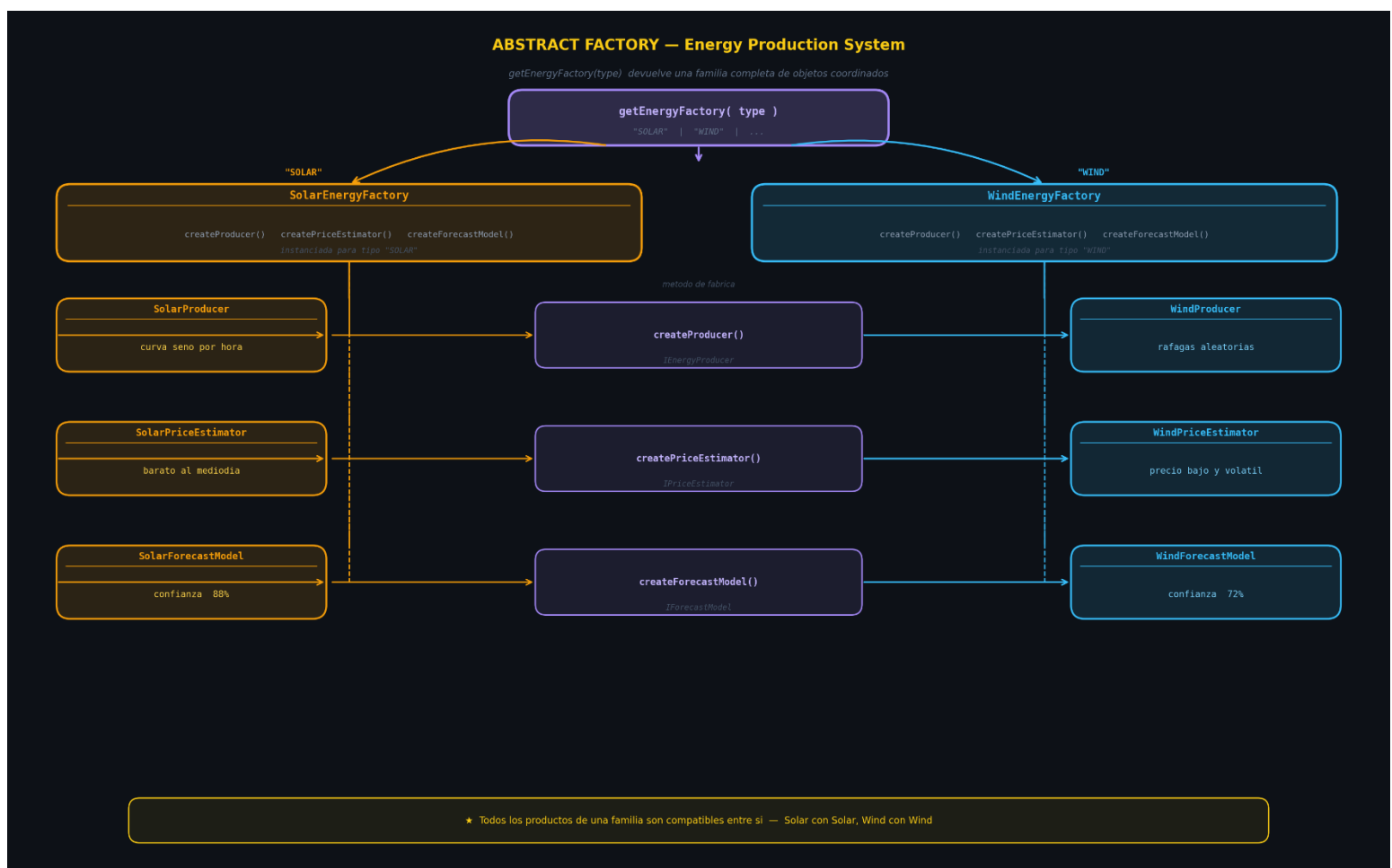
```
class SolarEnergyFactory implements IEnergySourceFactory {
  createProducer() { return new SolarProducer(); } // curva seno solar
  createPriceEstimator() { return new SolarPriceEstimator(); } // más barato al mediodía
  createForecastModel() { return new SolarForecastModel(); } // confianza 88%
}

class WindEnergyFactory ... // variación tipo ráfaga, confianza 72%
class BatteryEnergyFactory ... // carga/descarga, confianza 95%
class GridEnergyFactory ... // ilimitada, la más cara, confianza 99%
```

Por ultimo el selector de las cuatro clases de fabrica

```
export function getEnergyFactory(source: EnergySource): IEnergySourceFactory {  
  switch (source) {  
    case "SOLAR": return new SolarEnergyFactory();  
    case "WIND": return new WindEnergyFactory();  
    case "BATTERY": return new BatteryEnergyFactory();  
    case "GRID": return new GridEnergyFactory();  
  }  
}
```

Diagrama Uml



Patrón 4: Builder

¿Qué problema resuelve?

Una TradeOrder tiene muchos campos opcionales. Sin Builder, el constructor se convierte en un desastre:

```
// ✗ Sin Builder - imposible saber qué significa cada argumento  
new TradeOrder("SELL", 12.5, 0.124, "SOLAR", "FIXED", "URGENT", 30, true, false, "Excedente")
```

Con Builder se construye la orden **paso a paso**, solo con lo que se necesita :

```
new TradeOrderBuilder()  
  .ofType("SELL")  
  .withAmount(12.5)  
  .atPrice(0.124)  
  .fromSource("SOLAR")  
  .withPriority("URGENT")  
  .build()
```

¿Dónde se usa?

El patrón tiene **3 participantes** hasta el momento en el proyecto

1. El Producto — TradeOrder (interfaz)

El objeto complejo que se construye al final:

```
interface TradeOrder {  
  id: string;          type: OrderType;      amountKwh: number;  
  pricePerKwh: number; energySource: EnergySource;  
  pricingMode: PricingMode; priority: OrderPriority;  
  expiresAt: Date | null; note: string | null;  
  conditions: TradeOrderConditions;  
  maxSlippagePercent: number; totalValueUsd: number;  
  status: OrderStatus;  
}
```


2. El Builder — TradeOrderBuilder

Acumula el estado internamente y expone métodos encadenables:

```
class TradeOrderBuilder {
  // campos privados con valores por defecto seguros
  private _type: OrderType | null = null;
  private _amountKwh: number | null = null;
  private _priority: OrderPriority = "NORMAL"; // default
  private _conditions: TradeOrderConditions = {};
  // ...

  ofType(type)      { this._type = type;      return this; }
  withAmount(kwh)   { this._amountKwh = kwh;   return this; }
  atPrice(usd)      { this._pricePerKwh = usd;  return this; }
  fromSource(src)   { this._energySource = src; return this; }
  expiresInMinutes(min) { /* calcula Date */    return this; }
  requireGreenCertified() { this._conditions.requireGreenCertified = true; return this; }

  build(): TradeOrder {
    // Valida campos requeridos
    if (!this._type) throw new TradeOrderBuildError("El tipo es requerido");
    if (this._pricePerKwh <= 0) throw new TradeOrderBuildError("El precio debe ser > 0");
    // Calcula campos derivados y retorna el producto final
    return { id: `ORD-...`, totalValueUsd: kwh * price, status: "DRAFT", ... };
  }

  reset() { /* Limpia todo para reutilizar */ return this; }
}
```

3. El Director — TradeOrderDirector

Encapsula **recetas predefinidas** usando el Builder. Sabe qué pasos llamar y en qué orden:

```
class TradeOrderDirector {
  constructor(private builder: TradeOrderBuilder) {}

  buildUrgentSolarSell(kwh, price): TradeOrder {
    return this.builder.reset()
      .ofType("SELL").withAmount(kwh).atPrice(price)
      .fromSource("SOLAR").withPricingMode("DYNAMIC")
      .withPriority("URGENT").expiresInMinutes(30)
      .withMaxSlippage(5).allowPartialFill()
      .build();
  }

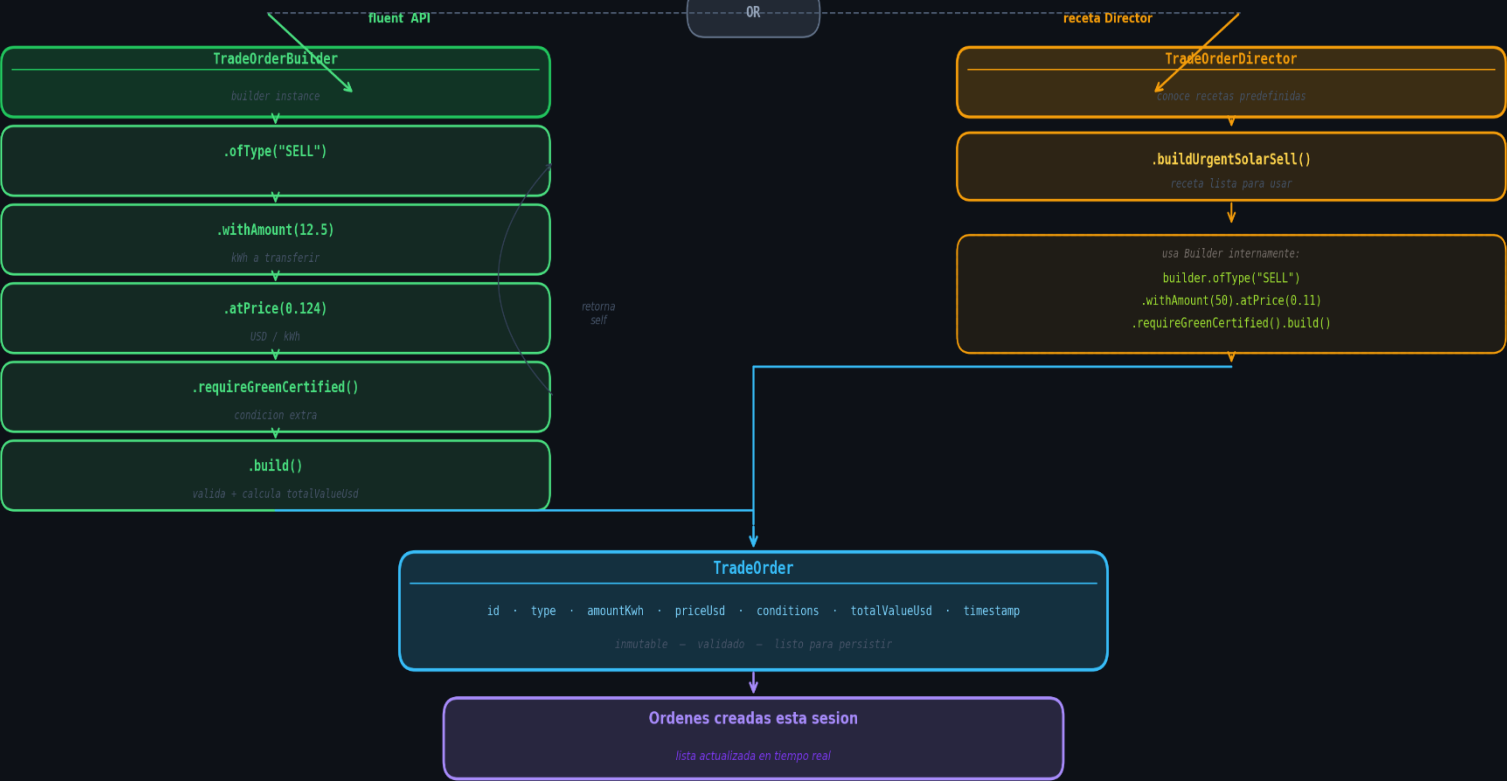
  buildScheduledGreenBuy(kwh, maxPrice): TradeOrder { ... }
  buildPeakBatteryDischarge(kwh, price): TradeOrder { ... }
}
```

Diagrama Uml

PATRON BUILDER — TradeOrder Construction

Dos rutas para construir la misma orden — fluent API o Director con receta

Usuario llena el formulario / hace clic en receta
campos: tipo, cantidad, precio, condiciones



LEYENDA

Usuario / entrada

Builder (fluent API)

Director (receta)

Producto final: TradeOrder

UI / vista

★ El Director es opcional — simplifica la creacion de objetos complejos con configuraciones frecuentes

Patrón 5: Prototype

¿Qué problema resuelve?

Un trader vende 10 kWh solares cada mañana. Sin Prototype, lo reconstruye con el Builder desde cero todos los días. Con Prototype, guarda esa orden como plantilla y la clona en segundos el clon es 100% independiente, tiene nuevo ID y puede pisar campos.

¿Dónde se usa?

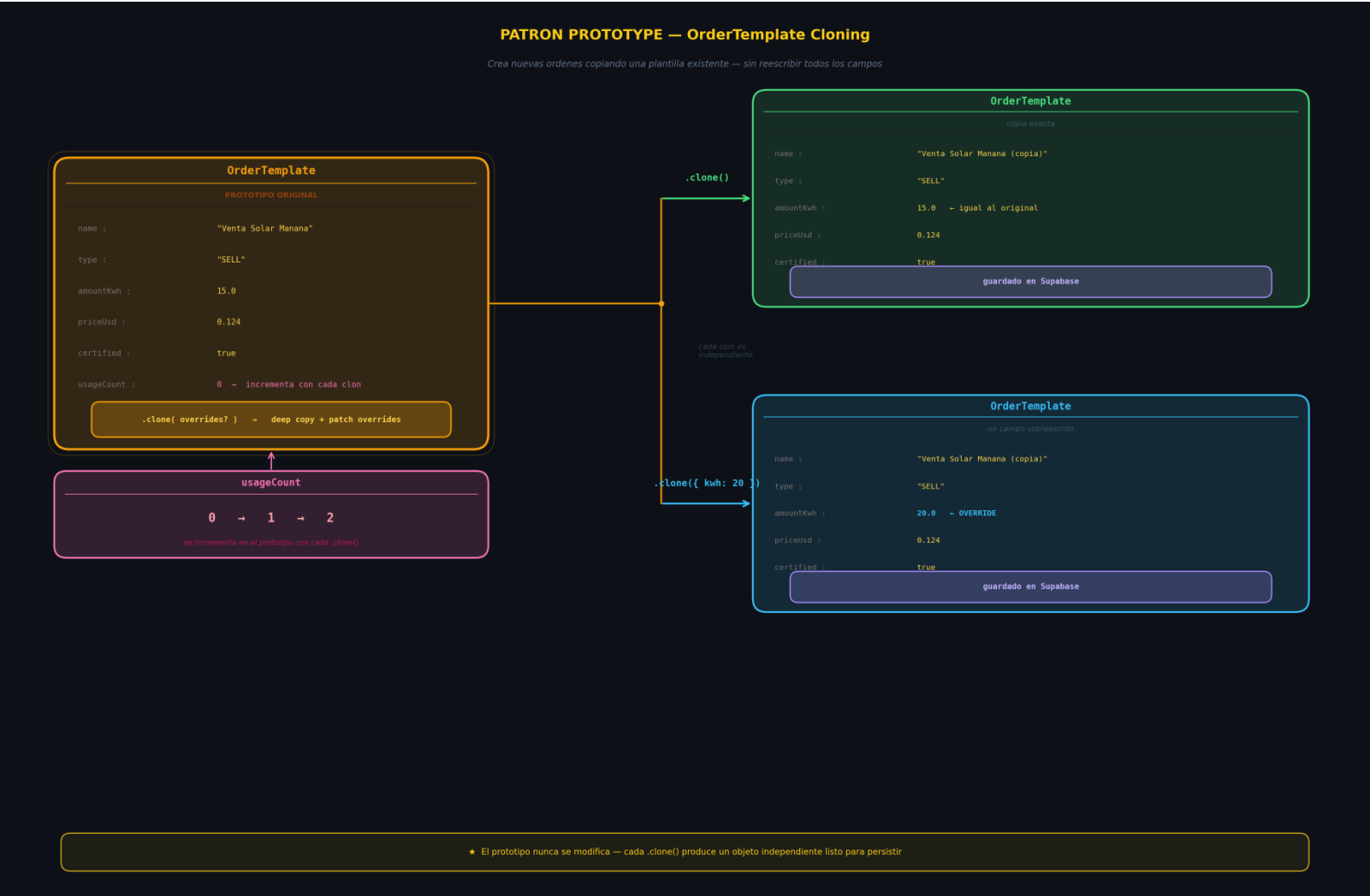
```
interface IOrderPrototype {  
  clone(): OrderTemplate;  
  getOrder(): TradeOrder;  
}
```

```
class OrderTemplate implements IOrderPrototype {  
  private readonly _order: TradeOrder; // orden congelada (immutable)  
  
  clone(overrides: OrderOverrides = {}): OrderTemplate {  
    this.usageCount++;  
  
    // Reconstruye con el Builder → nuevo ID, validación garantizada  
    const builder = new TradeOrderBuilder()  
      .ofType(this._order.type)  
      .withAmount(overrides.amountKwh ?? this._order.amountKwh) // ← override opcional  
      .atPrice(overrides.pricePerKwh ?? this._order.pricePerKwh)  
      .fromSource(this._order.energySource)  
      // ... copia todas las condiciones del original  
      .withNote(`Clon de: ${this.name}] ${this._order.note}`);  
  
    return new OrderTemplate(builder.build(), `${this.name} (copia)`);  
  }  
}
```

Registra las plantillas y las manda a la base de datos de supabase

```
class OrderTemplateRegistry {  
  // 3 templates pre-cargados: Venta Solar Mañana, Carga Batería Noche, Compra Verde  
  
  cloneFrom("Venta Solar Mañana", { amountKwh: 15 })  
  // → nueva TradeOrder: 15 kWh (override), precio/fuente/condiciones del original  
}
```

Diagrama Uml



PATRONES ESTRUCTURALES

Patrón 1: Adapter

¿Qué problema resuelve?

el Adapter traduce la interfaz incompatible de cada proveedor externo a la interfaz estándar IPriceEstimator. El resto del sistema (páginas, servicios) solo conoce IPriceEstimator y no sabe nada del proveedor real.

¿Dónde se usa?

1.Target: la interfaz que el sistema usa internamente:

```
interface IPriceEstimator {  
  getCurrentQuote(): Promise<PriceQuote>; // → siempre USD/kWh  
}
```

2 Adaptees: las interfaces incompatibles de cada proveedor:

```
interface IOmieMarketApi {  
  fetchDayAheadPrice(fecha: string): Promise<{  
    precio_mwh: number; // ← €/MWh (no USD/kWh)  
    tecnologia: "SOLAR" | "EOLICA" | "HIDRO" | "TERMICO"; // ← español  
    variacion_pct: number;  
  }>;  
}
```

3 Los Adapters: traducen el formato externo al interno:

```

class OmieAdapter implements IPriceEstimator { // + implementa Target
  constructor(private omieApi: IOmieMarketApi) {} // + envuelve al Adaptee

  async getCurrentQuote(): Promise<PriceQuote> {
    const raw = await this.omieApi.fetchDayAheadPrice(today);

    // TRADUCCIÓN: €/MWh → USD/kWh
    const priceUsdKwh = raw.precio_mwh * 1.08 * 0.001;

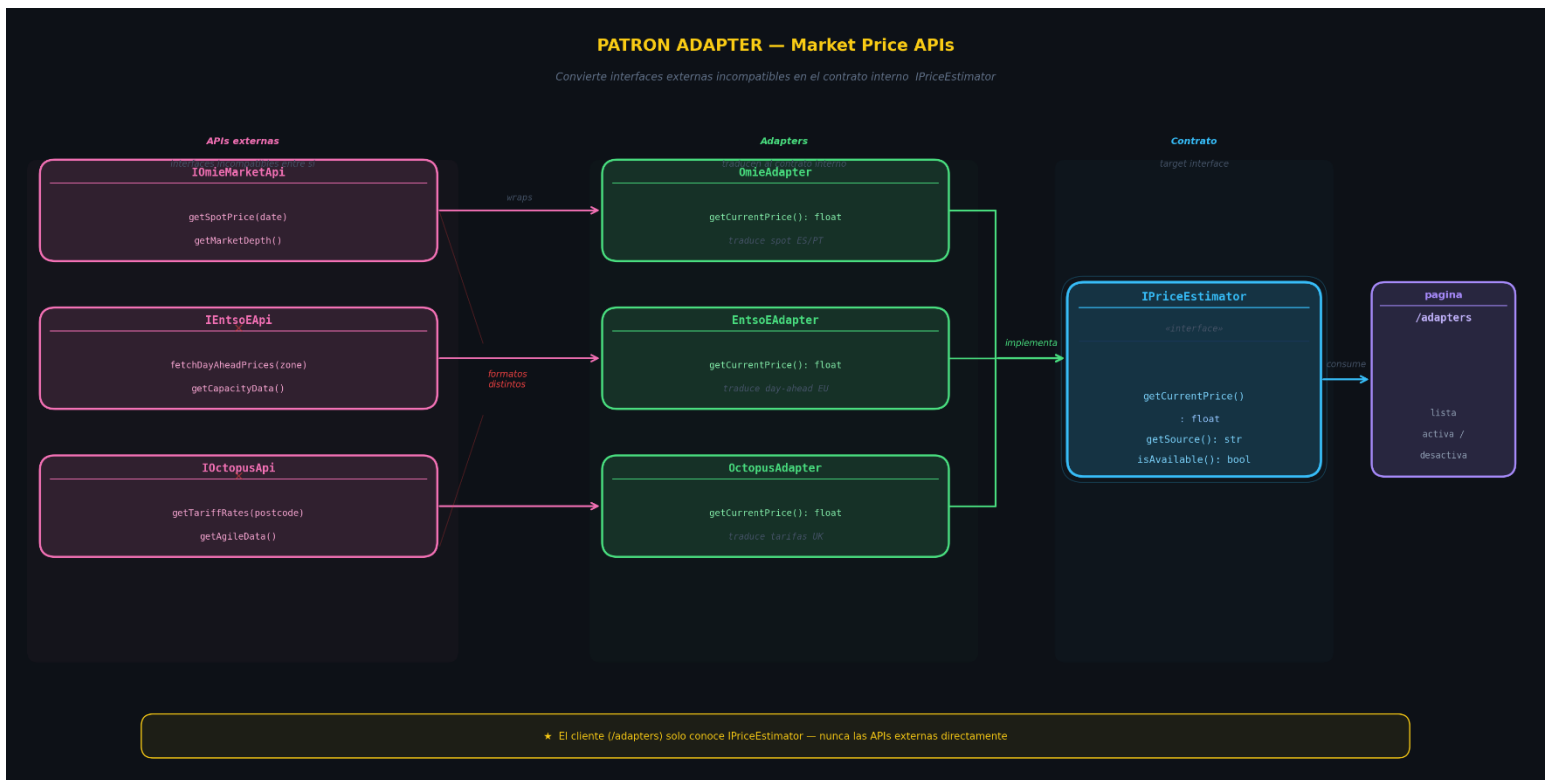
    // TRADUCCIÓN: "EOLICA" → "WIND"
    const source = { SOLAR:"SOLAR", EOLICA:"WIND", ... }[raw.tecnologia];

    // TRADUCCIÓN: variacion_pct → "UP"/"DOWN"/"STABLE"
    const trend = raw.variacion_pct > 1 ? "UP" : ...

    return { source, pricePerKwh: priceUsdKwh, trend, currency: "USD", ... };
  }
}

```

Diagrama Uml



Patrón 2: Bridge

¿Qué problema resuelve?

el sistema genera notificaciones de 3 tipos distintos (precio, orden, dispositivo) y necesita entregarlas por múltiples canales (consola, in-app, email, webhook). Sin Bridge, habría $3 \times 4 = 12$ clases Con Bridge: $3 + 4 = 7$ clases

¿Dónde se usa?

IMPLEMENTOR — la jerarquía de cómo se entrega:

```
interface INotificationChannel {  
    deliver(notification: Notification): Promise<void>;  
}  
  
class ConsoleChannel implements INotificationChannel { ... } //  
imprime en terminal  
class InAppChannel implements INotificationChannel { ... } //  
guarda en memoria → UI  
class EmailChannel implements INotificationChannel { ... } //  
simula SendGrid  
class WebhookChannel implements INotificationChannel { ... } //  
simula POST HTTP
```

ABSTRACTION — la clase base que compone el canal (no hereda):

```
abstract class Notifier {  
    constructor(protected channel: INotificationChannel) {} // ← el  
    "bridge"  
  
    setChannel(channel: INotificationChannel) { // cambia canal en  
        runtime  
        this.channel = channel;  
    }  
}
```

REFINED ABSTRACTIONS — la jerarquía de qué se notifica:


```

class PriceAlertNotifier extends Notifier {
  async alertPriceSpike(source, price, threshold) {
    // formatea el mensaje específico de precio...
    await this.channel.deliver({ title: "Precio alto", level:
"CRITICAL", ... });
  }
}

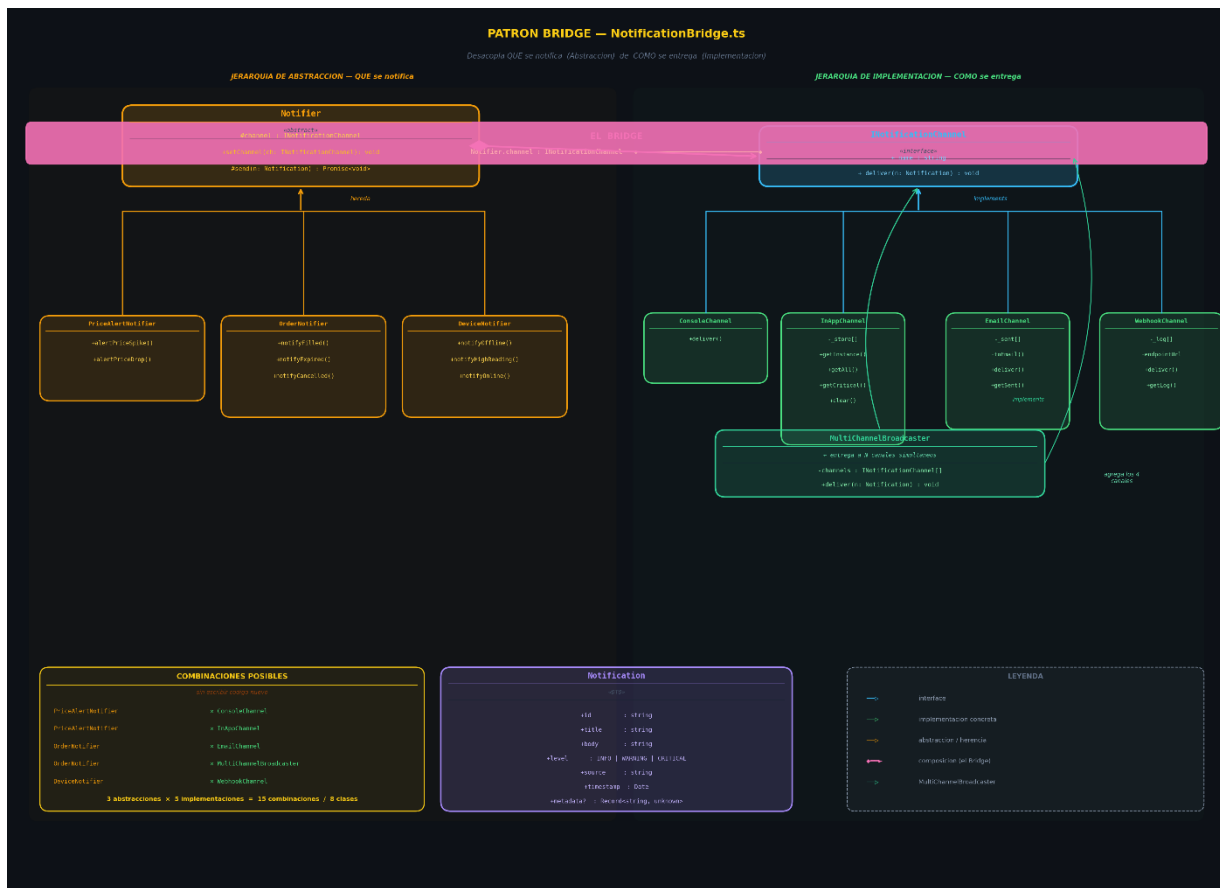
class OrderNotifier extends Notifier {
  async notifyOrderFilled(orderId, kwh, price) { ... }
  async notifyOrderExpired(orderId, reason) { ... }
}

class DeviceNotifier extends Notifier {
  async notifyDeviceOffline(deviceId, name) { ... }
}

```

El "Bridge" es la referencia **protected channel** en la clase base. Al cambiar el canal, todo el comportamiento de entrega cambia sin tocar ninguna de las 3 clases de notificación:

Diagrama Uml



Patrón 3: Decorator

¿Qué problema resuelve?

El OmieAdapter (del patrón Adapter) obtiene precios, pero falta:

- ¿Cómo evitar llamar la API 60 veces por minuto? → **Caché**
- ¿Cómo saber si los precios subieron de golpe? → **Alertas**
- ¿Cómo registrar cada consulta para análisis? → **Logging**
- ¿Qué pasa si la red falla? → **Reintentos**

Sin Decorator: habría que meter todo eso dentro del OmieAdapter, mezclando responsabilidades. Con Decorator: **cada comportamiento es una capa independiente y apilable.**

¿Dónde se usa?

COMPONENT — la interfaz del patrón Adapter:

```
interface IPriceEstimator {  
  getCurrentQuote(): Promise<PriceQuote>;  
}
```

DECORATOR BASE — envuelve cualquier IPriceEstimator:

```
abstract class PriceEstimatorDecorator implements IPriceEstimator {  
  constructor(protected readonly wrapped: IPriceEstimator) {}  
  
  getCurrentQuote(): Promise<PriceQuote> {  
    return this.wrapped.getCurrentQuote(); // delega por defecto  
  }  
}
```

4 DECORADORES CONCRETOS — cada uno añade una sola responsabilidad:

```

class CachingDecorator extends PriceEstimatorDecorator {
  async getCurrentQuote() {
    if (cache valida) return this._cached; // corto-circuito
    const quote = await this.wrapped.getCurrentQuote(); // delega y cachea
    this._cached = quote;
    return quote;
  }
  getStats() → { hits, misses, ratio }
}

class LoggingDecorator extends PriceEstimatorDecorator {
  async getCurrentQuote() {
    const start = Date.now();
    const quote = await this.wrapped.getCurrentQuote();
    this._history.unshift({ price, durationMs, fromCache, ... });
    return quote;
  }
}

class AlertingDecorator extends PriceEstimatorDecorator {
  async getCurrentQuote() {
    const quote = await this.wrapped.getCurrentQuote();
    if (quote.pricePerKwh > this.spikeThreshold) {
      // USA EL BRIDGE para entregar la alerta + integración entre patrones
      new PriceAlertNotifier(InAppChannel.getInstance()).alertPriceSpike(...);
    }
    return quote;
  }
}

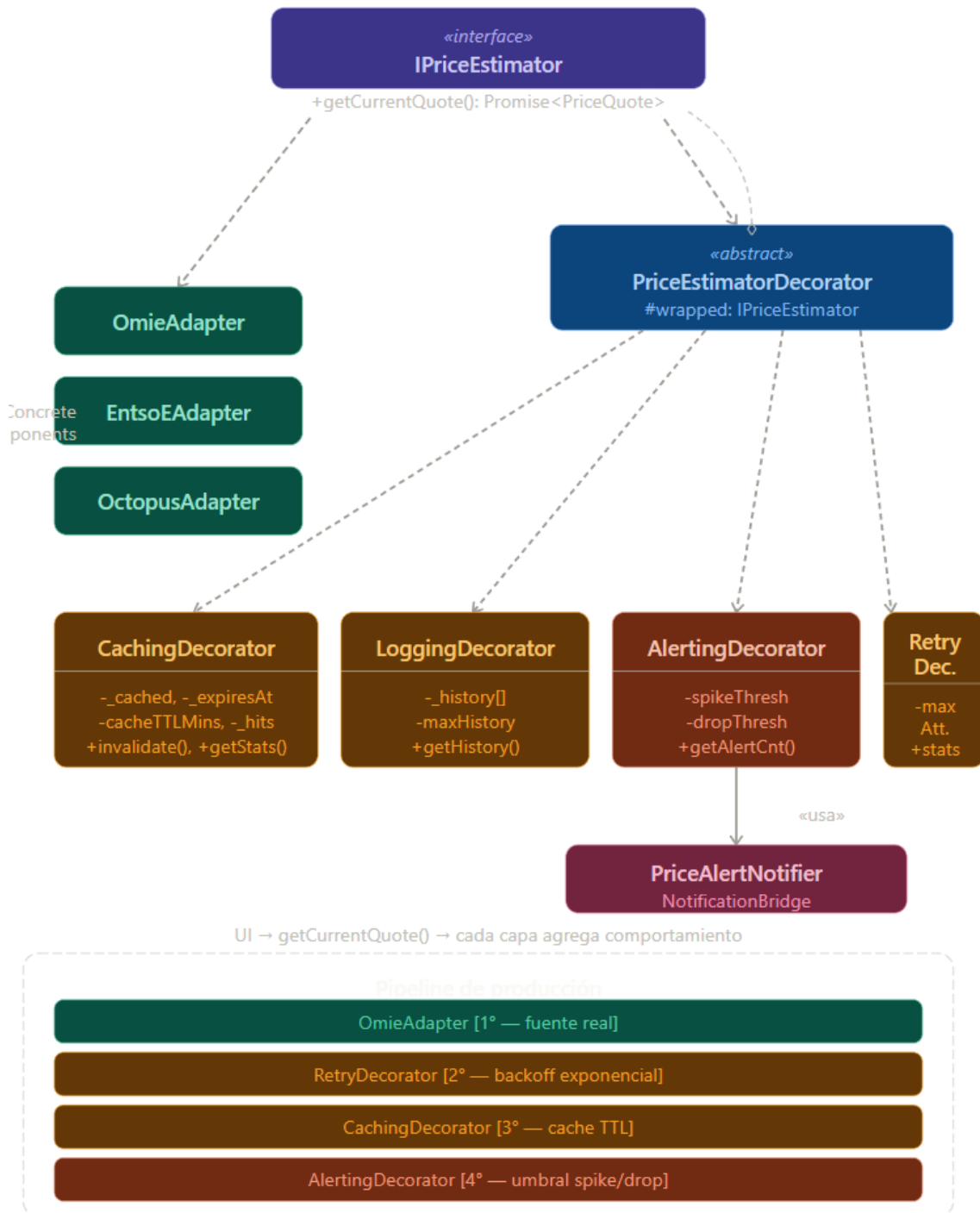
class RetryDecorator extends PriceEstimatorDecorator {
  async getCurrentQuote() {
    for (let i = 1; i <= maxAttempts; i++) {
      try { return await this.wrapped.getCurrentQuote(); }
      catch { await sleep(baseDelay * 2^i); } // backoff exponencial
    }
  }
}

```

Pipeline completo — los decoradores se apilan de adentro hacia afuera:

```
const pipeline = buildPricePipeline(new OmieAdapter(api), {  
  cacheTTLMinutes: 1,  
  spikeThreshold: 0.18,  
  maxRetries: 3,  
});  
  
// Capa más externa → Logging  
//   → Alerting  
//     → Caching  
//       → Retry  
//         → OmieAdapter (capa más interna)
```

Diagrama Uml



Patrón 4: Facade

¿Qué problema resuelve?

Antes de la Facade, para que una página cree y publique una orden necesitaba conocer y coordinar 5 subsistemas por separado.

```
// ❌ Sin Facade - el cliente debe conocer TODOS los subsistemas:  
const builder = new TradeOrderBuilder(); // subsistema 1  
const order = builder.ofType("SELL")...build();  
const db = getSupabaseAdmin(); // subsistema 2  
await db.from("energy_orders").insert({...});  
const notifier = new OrderNotifier(InAppChannel.getInstance()); // subsistema 3  
await notifier.notifyOrderFilled(order.id, ...);
```

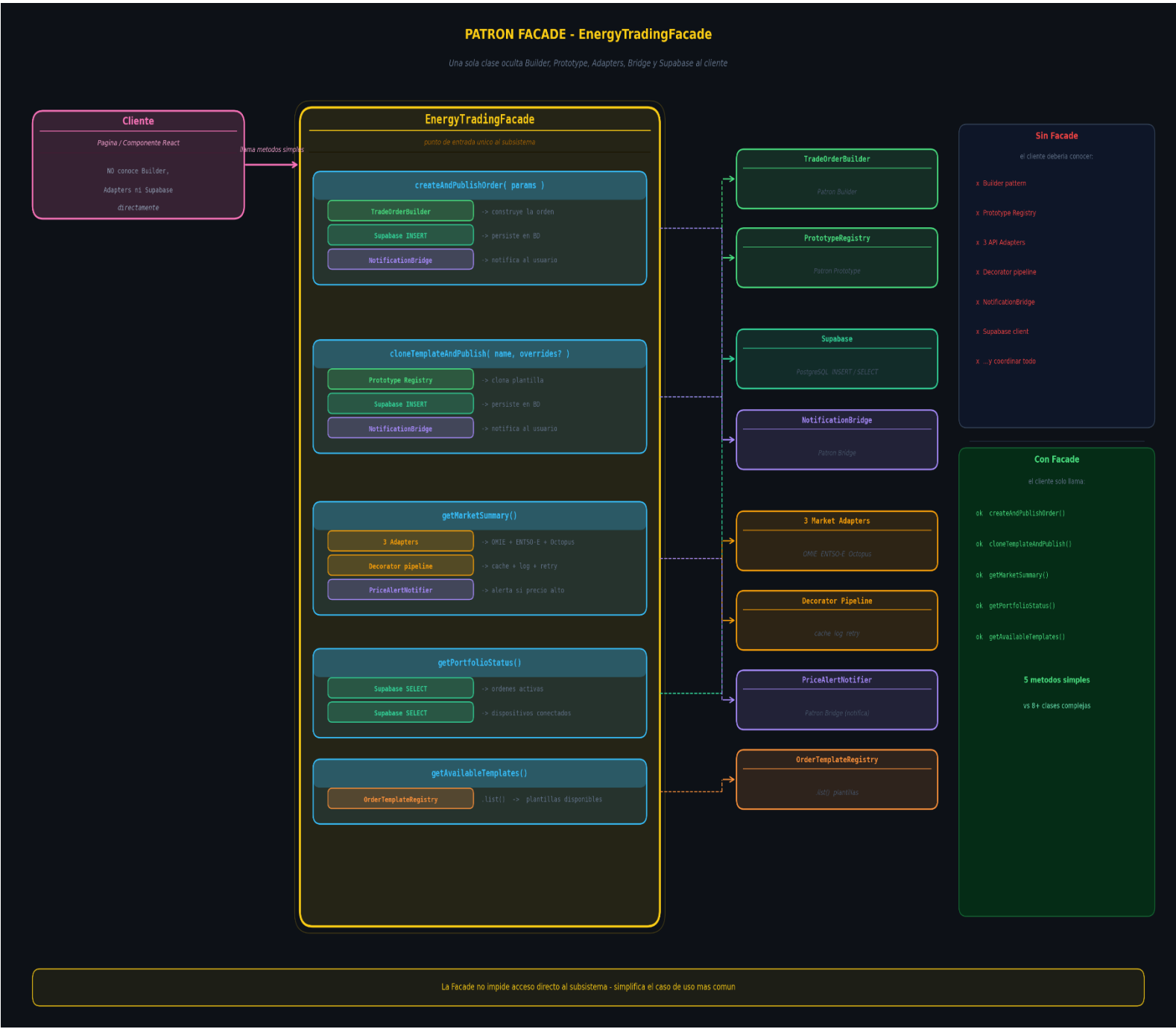
EnergyTradingFacade expone operaciones de alto nivel que internamente coordinan todos los subsistemas. El cliente solo llama a la Facade y obtiene el resultado.

¿Dónde se usa?

La Facade expone 5 operaciones de alto nivel, cada una orquestando subsistemas distintos:

```
async createAndPublishOrder(params: CreateOrderParams): Promise<FacadeResult<string>> {  
  // Subsistema 1 - Builder  
  const order = new TradeOrderBuilder().ofType(params.type)...build();  
  
  // Subsistema 2 - Supabase  
  const { error } = await getSupabaseAdmin().from("energy_orders").insert({...});  
  
  // Subsistema 3 - Bridge de notificaciones  
  await new OrderNotifier(InAppChannel.getInstance()).notifyOrderFilled(...);  
  
  return { ok: true, data: order.id };  
}
```

Diagrama Uml



Patrón 5: composite

¿Qué problema resuelve?

Una red de energía tiene dispositivos individuales y grupos de dispositivos. Sin Composite, el código tiene que preguntar constantemente "¿es un dispositivo o es un grupo?"

```
// ❌ Sin Composite - Lógica diferente para cada caso:
function calcularTotal(nodo: unknown): number {
  if (nodo instanceof EnergyDevice) {
    return nodo.energyKwh; // caso hoja
  } else if (nodo instanceof EnergyGroup) {
    return nodo.children.reduce((sum, c) => sum + calcularTotal(c), 0); // caso grupo
  }
  return 0;
}
```

Con Composite, ambos implementan IEnergyNode y el cliente llama siempre lo mismo

```
// ✅ Con Composite - el cliente no sabe si es hoja o grupo:
unPanel.getTotalKwh() // → 42.5 kWh
unaGranja.getTotalKwh() // → 91.8 kWh (suma recursiva de 3 paneles)
elPortfolio.getTotalKwh() // → 593.8 kWh (suma de toda la jerarquía)
```

¿Dónde se usa?

Component — la interfaz común

```
interface IEnergyNode {
  getTotalKwh(): number; // energía total
  getPeakPowerKw(): number; // potencia pico
  getStatus(): NodeStatus; // ONLINE | PARTIAL | OFFLINE
  getChildCount(): number; // 0 si Leaf, N si Composite
  describe(): string; // árbol de texto
}
```

Leaf — un dispositivo físico individual:

```
class EnergyDevice implements IEnergyNode {
  getTotalKwh() → this.snap.energyKwh // su propio valor
  getPeakPowerKw() → this.snap.peakPowerKw // su propio valor
  getChildCount() → 0 // siempre cero
}
```

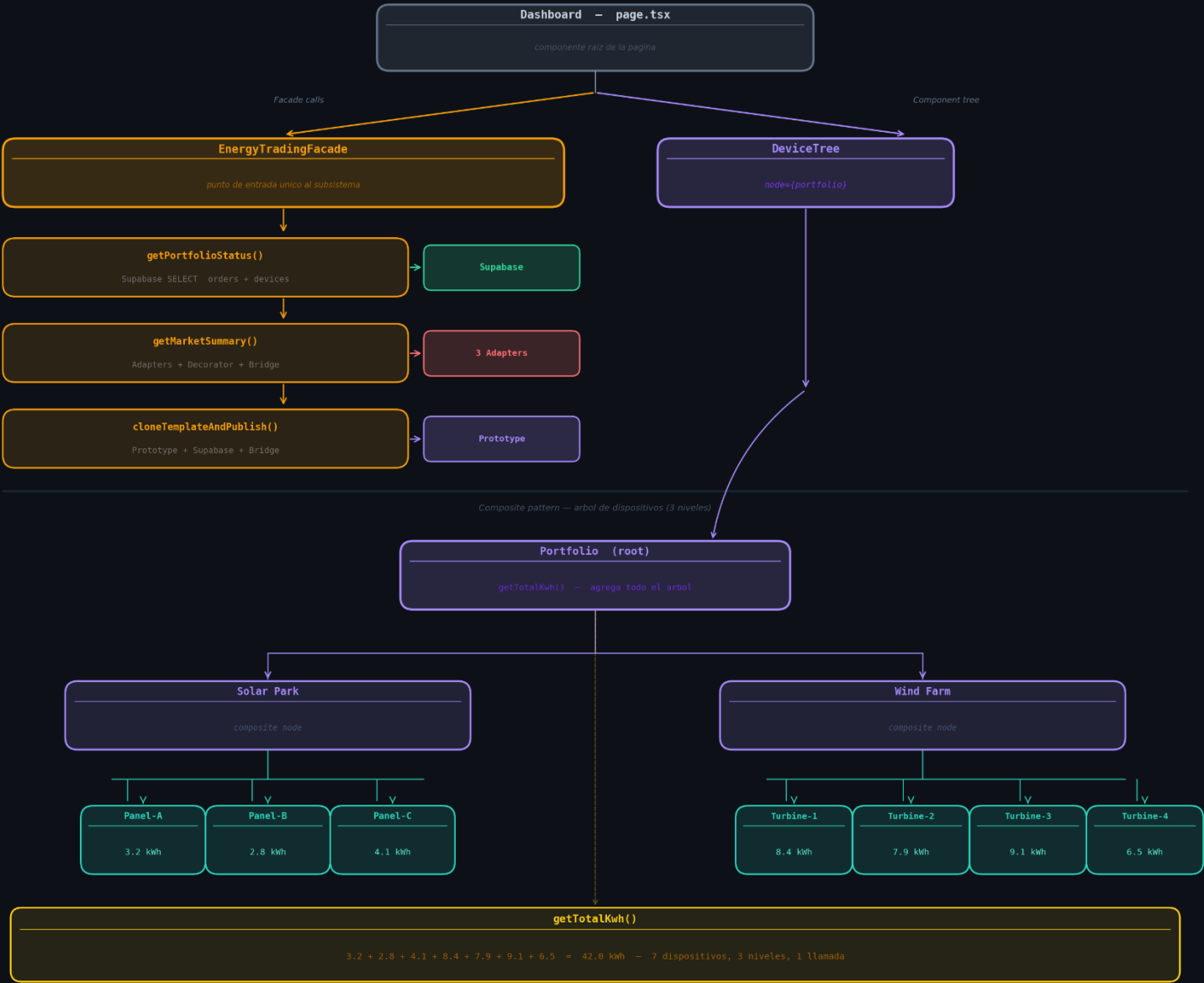
Composite — un grupo que contiene otros nodos:


```
class EnergyGroup implements IEnergyNode {  
    private _children: IEnergyNode[];    // puede ser Leaf u otro EnergyGroup  
  
    // RECURSIVIDAD - delega a cada hijo, sin saber si es Leaf o Group  
    getTotalKwh()    → this._children.reduce((sum, c) => sum + c.getTotalKwh(), 0)  
    getPeakPowerKw() → this._children.reduce((sum, c) => sum + c.getPeakPowerKw(), 0)  
  
    // ONLINE si todos online; PARTIAL si alguno; OFFLINE si ninguno  
    getStatus() → allOnline ? "ONLINE" : anyOnline ? "PARTIAL" : "OFFLINE"  
  
    // Cuenta todos los nodos en el subárbol (recursivo)  
    getChildCount() → children.reduce((sum, c) => sum + 1 + c.getChildCount(), 0)  
}
```

Diagrama Uml

FACADE + COMPOSITE — Dashboard Architecture

page.tsx coordina Facade para operaciones y Composite para el arbol de dispositivos



CLAVE DEL DIAGRAMA

- page.tsx / Dashboard**
componente React raiz, orquesta todo
- Supabase**
PostgreSQL — INSERT ordenes / SELECT portfolio
- Composite (Portfolio)**
arbol de dispositivos — leaf y branch misma interfaz
- EnergyTradingFacade**
oculta complejidad: Builder, Adapters, Prototype, Bridge
- 3 Adapters + Decorator**
OMIE, ENT50-E, Octopus con cache, log y retry
- Leaf devices (7 total)**
Panel-A/B/C, Turbine-1/2/3/4 — cada uno retorna kWh

Patrón 6: Flyweight

¿Qué problema resuelve?

Una red IoT con 50 dispositivos genera ~3000 lecturas/minuto. Sin Flyweight, cada objeto lectura almacena los mismos 5 campos repetidos en memoria:

```
Lectura #1 → { deviceId:"D1", timestamp:..., value:12.5, sensorType:"POWER", unit:"kW",
label:"Potencia", color:"#f59e0b", range:{0-50} }
Lectura #2 → { deviceId:"D1", timestamp:..., value:11.8, sensorType:"POWER", unit:"kW",
label:"Potencia", color:"#f59e0b", range:{0-50} }
Lectura #3 → { deviceId:"D2", timestamp:..., value:13.1, sensorType:"POWER", unit:"kW",
label:"Potencia", color:"#f59e0b", range:{0-50} }
...10,000 lecturas con los mismos 5 campos copiados ❌
```

Con Flyweight: esos 5 campos se almacenan una sola vez, compartidos:

```
Lectura #1 → { deviceId:"D1", timestamp:..., value:12.5, flyweight → ▶ }
Lectura #2 → { deviceId:"D1", timestamp:..., value:11.8, flyweight → ▶ }
Lectura #3 → { deviceId:"D2", timestamp:..., value:13.1, flyweight → ▶ }
                                                    ▶ { unit:"kW",
label:"Potencia", color:"#f59e0b", range:{0-50} }
...10,000 lecturas compartiendo 1 sola instancia ✅
```

¿Dónde se usa?

Estado intrínseco (compartido, inmutable) — `SensorTypeFlyweight`

```
// 1 sola instancia por tipo de sensor para toda la app
interface SensorTypeFlyweight {
    readonly sensorType: SensorType; // "POWER"
    readonly unit: string; // "kW"
    readonly displayLabel: string; // "Potencia"
    readonly colorHex: string; // "#f59e0b"
    readonly normalRange: { min, max }; // { min:0, max:50 }
    format(value): string; // "12.50 kW"
    classify(value): "NORMAL"|"HIGH"|"CRITICAL";
}
```

Fábrica / Pool — `ReadingFlyweightFactory`:

```

class ReadingFlyweightFactory {
    private static readonly _pool = new Map<SensorType, SensorTypeFlyweight>();
    // Pre-carga los 8 tipos al arrancar el módulo

    static get(sensorType: SensorType): SensorTypeFlyweight {
        // Si ya existe → devuelve el mismo objeto (cache hit)
        // Si no → crea uno nuevo y lo guarda en el pool
    }

    static getStats() → { poolSize:8, totalRequests, cacheHits, hitRatio }
}

```

Estado extrínseco (único por lectura) — IoTReading:

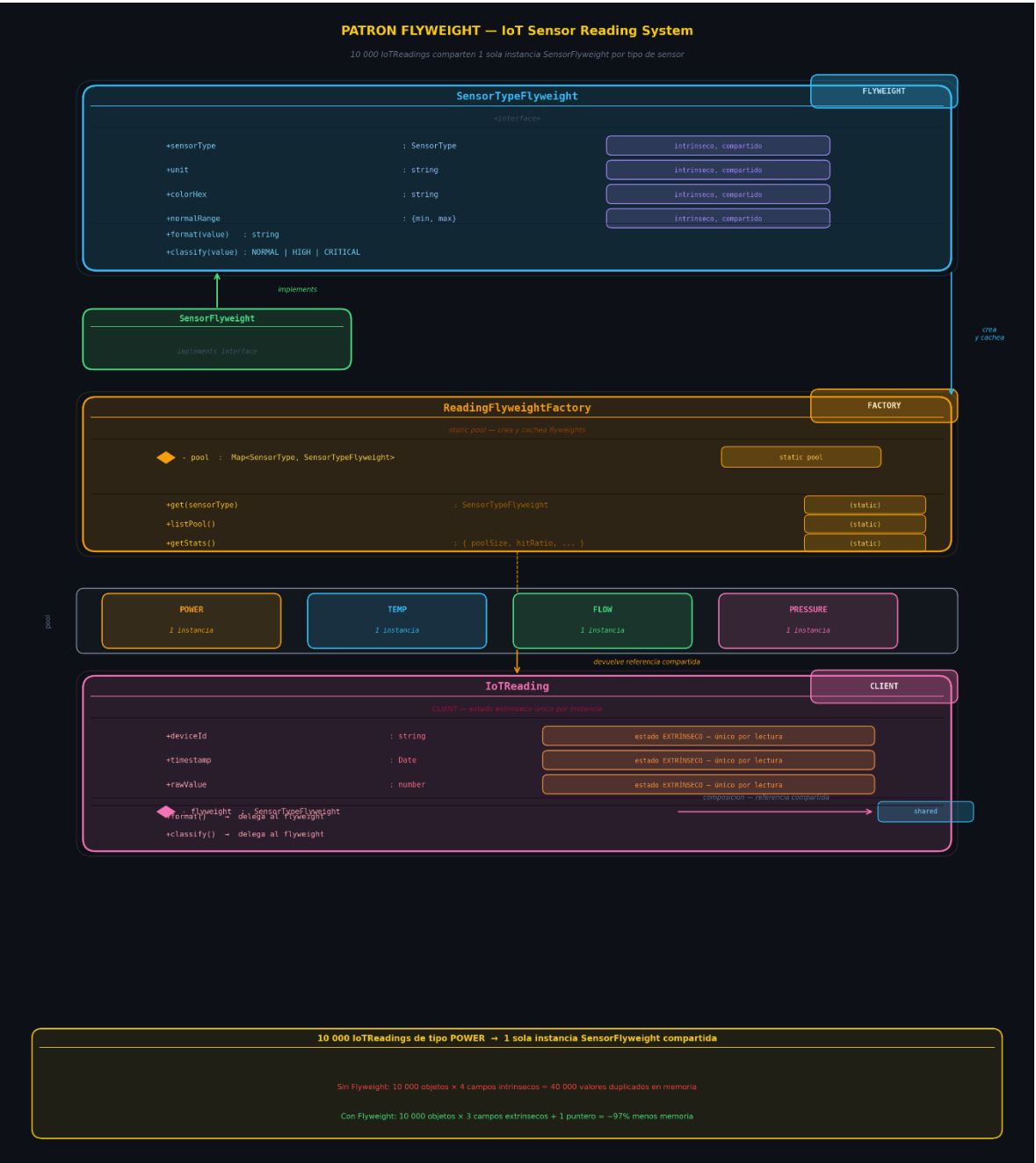
```

class IoTReading {
    readonly deviceId: string; // ← único
    readonly timestamp: Date; // ← único
    readonly rawValue: number; // ← único
    private _flyweight: SensorTypeFlyweight; // ← referencia compartida

    format() → delega al flyweight → "12.50 kW"
    classify() → delega al flyweight → "NORMAL"
}

```

Diagrama Uml



Patrón 7: Flyweight

¿Qué problema resuelve?

El acceso directo a Supabase desde las páginas IoT no tiene control de ningún tipo:

```
// ❌ Sin Proxy – acceso directo, sin seguridad ni caché:  
const { data } = await supabase.from("iot_devices").select("*"); // Llama BD en cada render  
await supabase.from("iot_devices").delete().eq("id", id); // cualquiera puede borrar  
// No hay log, no hay caché, no hay control de roles
```

Con Proxy, el cliente llama a la misma interfaz, pero el proxy intercepta:

```
// ✅ Con Proxy – el cliente no cambia, pero ahora hay control:  
const service = createDeviceService(userId, "VIEWER"); // VIEWER no puede borrar  
const devices = await service.getDevices(userId); // cacheado 30 segundos  
await service.deleteDevice("D1"); // → Error: "ADMIN requerido" (ni llega a Supabase)
```

¿Dónde se usa?

createDeviceService(userId, role) devuelve el proxy configurado Facade puede usarlo: EnergyTradingFacade en lugar de llamar Supabase directo, llama createDeviceService() La página IoT puede reemplazar sus llamadas directas con el proxy

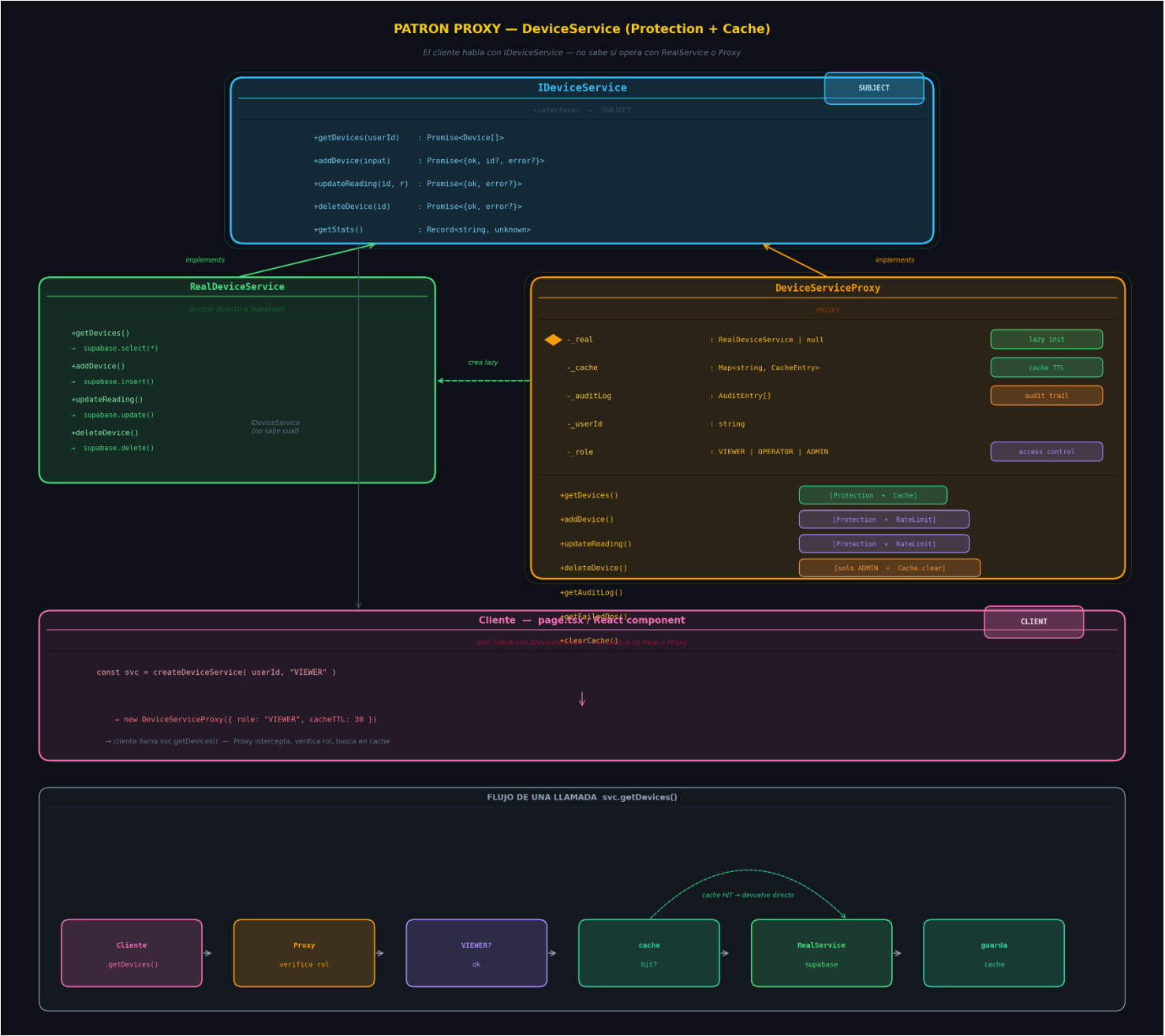
4 tipos de Proxy combinados en una sola clase:

Tipo	Qué hace
Virtual	Crea <code>RealDeviceService</code> solo cuando se llama por primera vez
Protection	Roles <code>VIEWER / OPERATOR / ADMIN</code> — cada operación requiere nivel mínimo
Caching	<code>getDevices()</code> cacheada N segundos; se invalida al agregar/borrar
Logging	Audita cada operación: quién, cuándo, duración, éxito/fallo

Jerarquía de roles:

```
VIEWER → solo getDevices()  
OPERATOR → + addDevice(), updateReading()  
ADMIN → + deleteDevice()
```

Diagrama Uml



PATRON QUE NO USARIA EN MI PROYECTO

Interpreter

¿Por qué no?

El punto clave es que aquí no hay nada que interpretar: no existe un lenguaje, no hay gramática, no hay ambigüedad que resolver. Introducir el patrón Interpreter en este contexto implica construir toda una infraestructura (tokens, árbol de sintaxis, clases por regla) para procesar algo que ya viene estructurado y explícito. Es, en esencia, simular un lenguaje donde no lo hay.

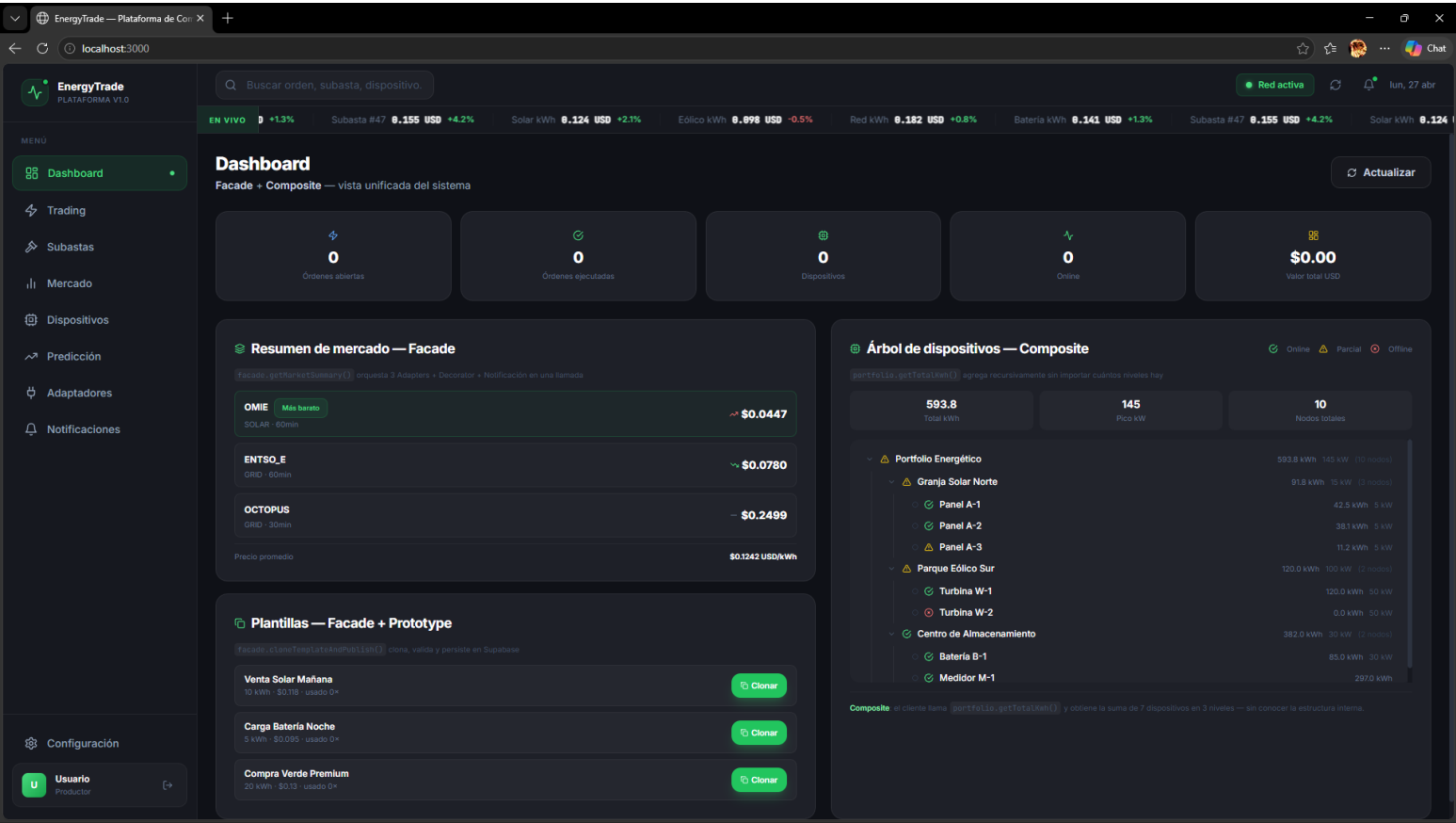
Esto no solo añade complejidad innecesaria, sino que además desplaza el problema hacia un terreno más difícil de mantener. Cada cambio en la “lógica” implica tocar múltiples clases en lugar de modificar una estructura de datos clara o una función directa. Lo que podría resolverse con configuraciones declarativas (JSON) y lógica sencilla termina convertido en un mini compilador interno sin justificación real.

Por eso, más que un patrón inútil, Interpreter es un patrón hiperespecializado que se vuelve contraproducente cuando se aplica fuera de su contexto natural. En proyectos modernos, su uso suele ser una señal de sobreingeniería más que de buen diseño.

FUNCIONAMIENTO DE LA UI DE MI PROYECTO

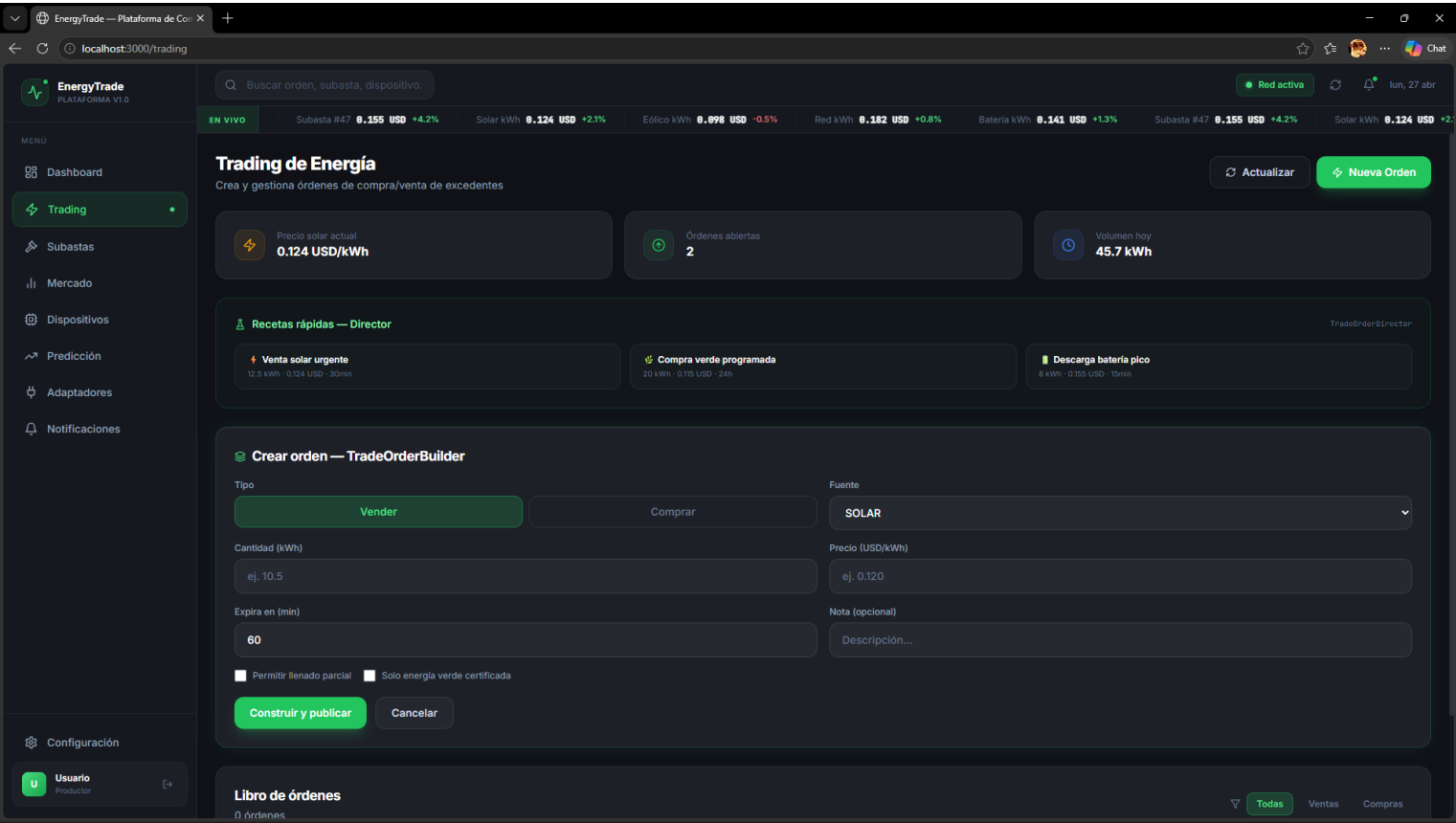
Dashboard

en este se puede ver con facade una vista unificada del sistema del usuario demo



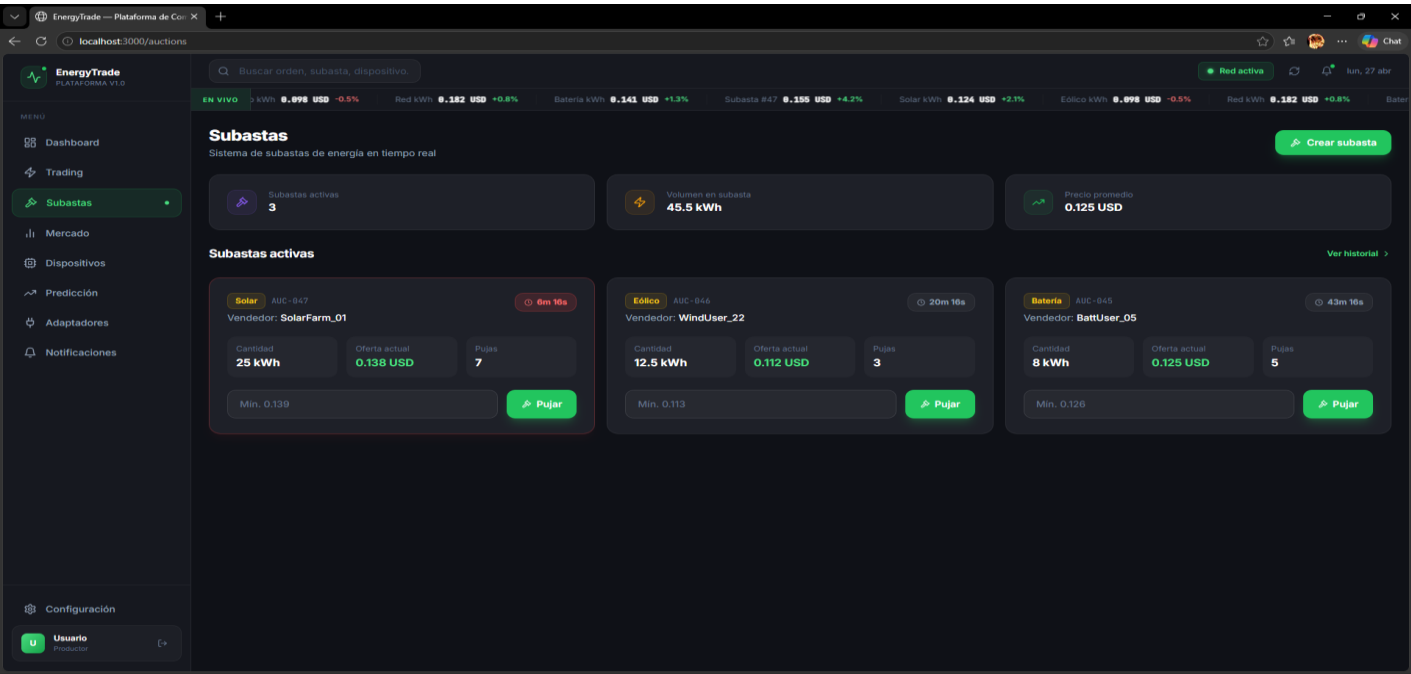
Trading

gestiona las órdenes de compra y venta



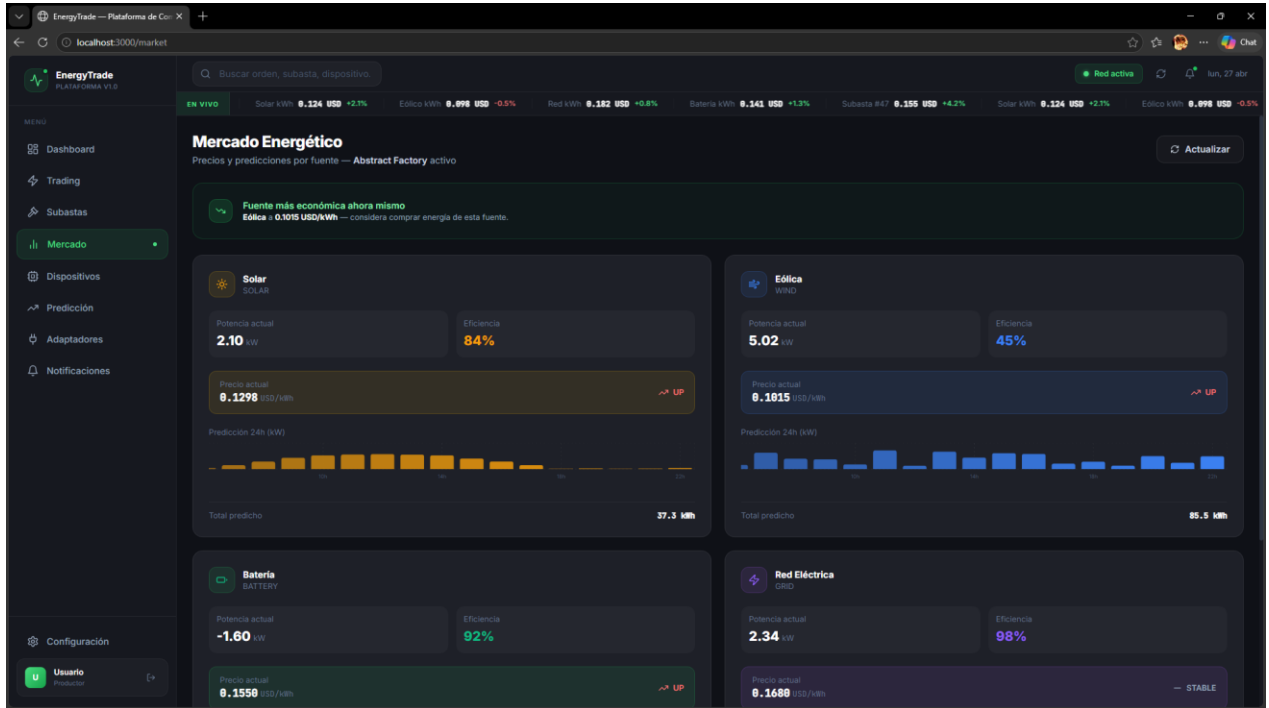
Subastas

Sistema de subasta de energía en tiempo real



Mercado

Muestra el mercado energético de los cuatro tipos de fuentes de energía manejadas



Dispositivos iot

Muestra los dispositivos iot y permite crearlos y añadirlos a una familia

Nuevo dispositivo IoT

Nombre *

ej. Panel Solar Terraza

Tipo

Panel Solar

Protocolo

Simulado (desarrollo)

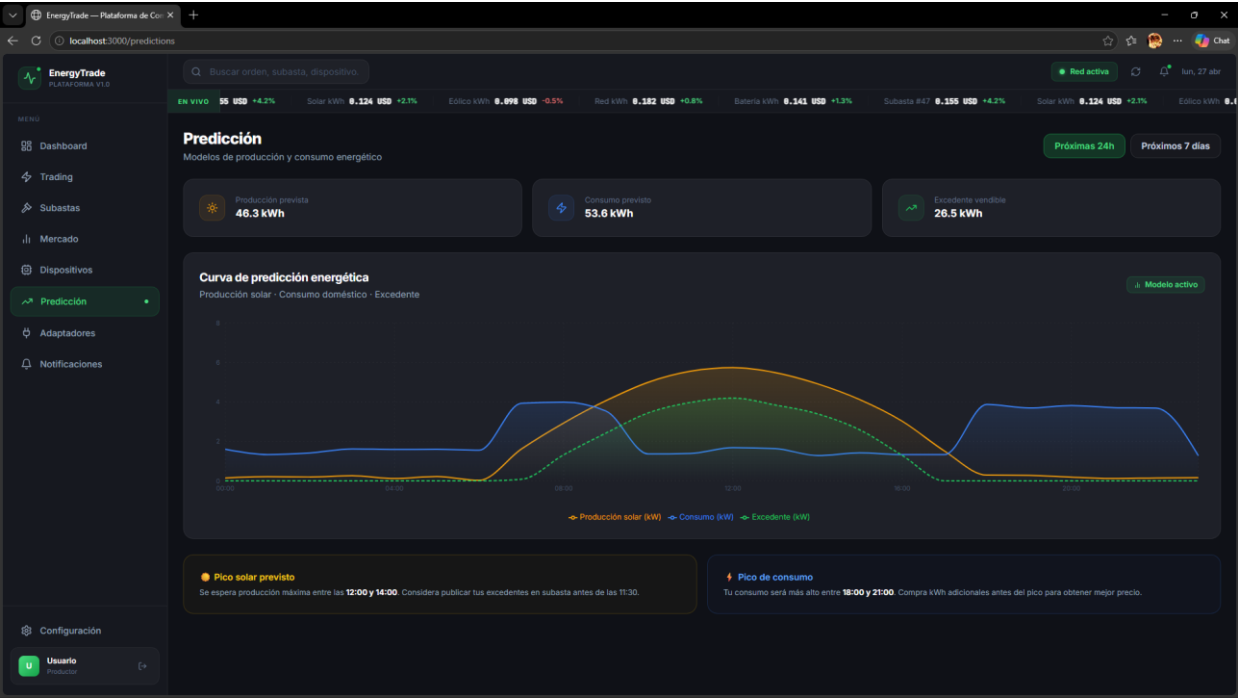
Ubicación (opcional)

ej. Techo Norte

+ Guardar en Supabase Cancelar

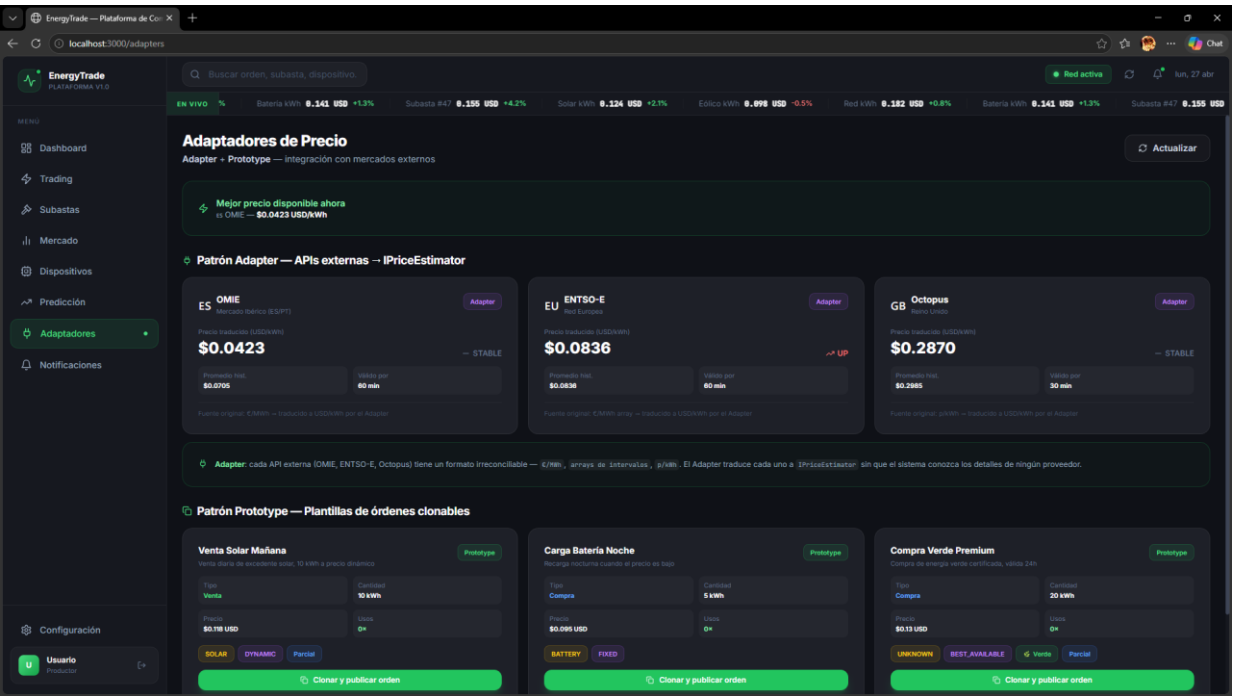
Predicción

Muestra una estimación del consumo versus el consumo



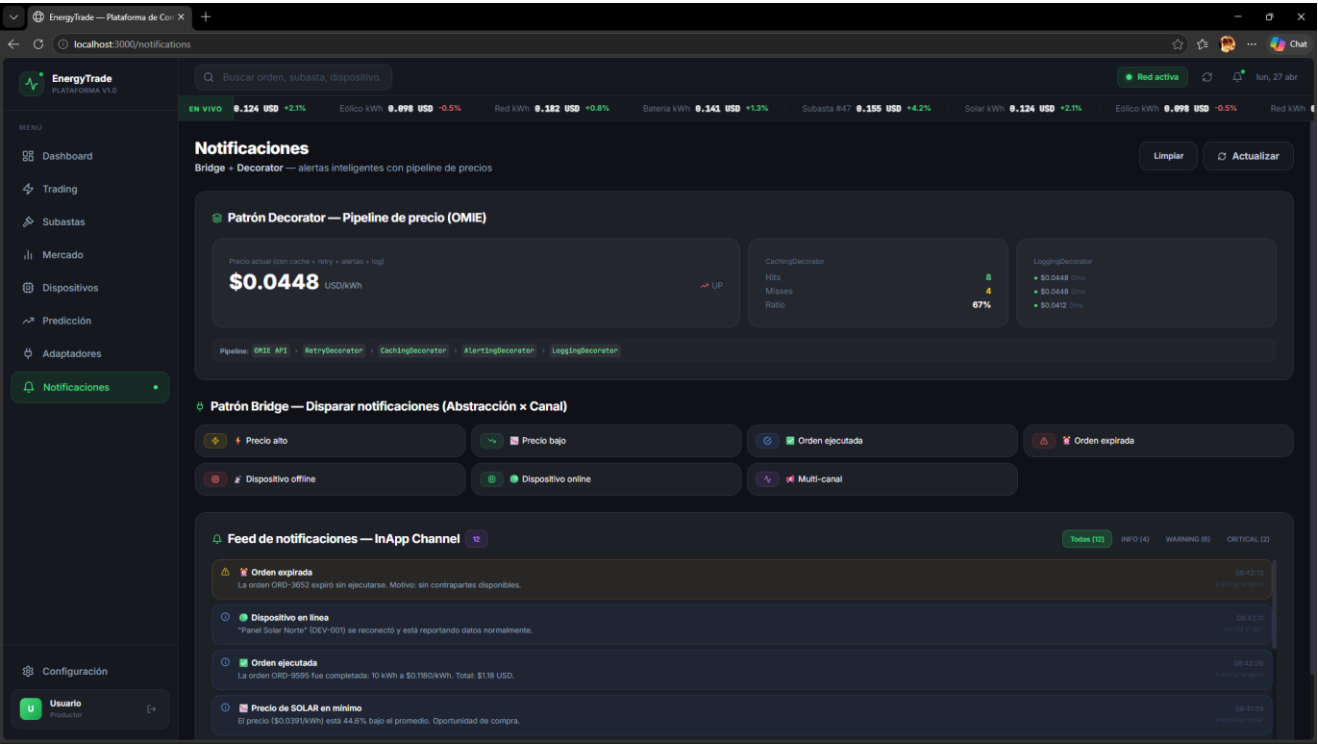
Adaptadores de precio

Adaptadores de los diferentes tipos de precio de energía según la región de la red



Notificaciones

Notifica y alerta todas las novedades que tiene el usuario en diferentes tipos de canales



PATRONES DE COMPORTAMIENTO

Patrón 1: Strategy

¿Qué problema resuelve?

El matching de órdenes de compra/venta de energía necesita algoritmos distintos según el contexto. Sin Strategy, el código sería un bloque gigante con if/else

```
// ✗ Sin Strategy - código frágil, imposible de extender:
function matchOrders(orders, mode) {
  if (mode === "price")      { /* 30 líneas */ }
  else if (mode === "green") { /* 30 líneas */ }
  else if (mode === "fifo")  { /* 30 líneas */ }
  // Para agregar una estrategia nueva → modificar esta función ✗
}
```

Con Strategy, cada algoritmo vive en su propia clase, el Context las intercambia en runtime, y agregar una nueva no toca ningún código existente.

¿Dónde se usa?

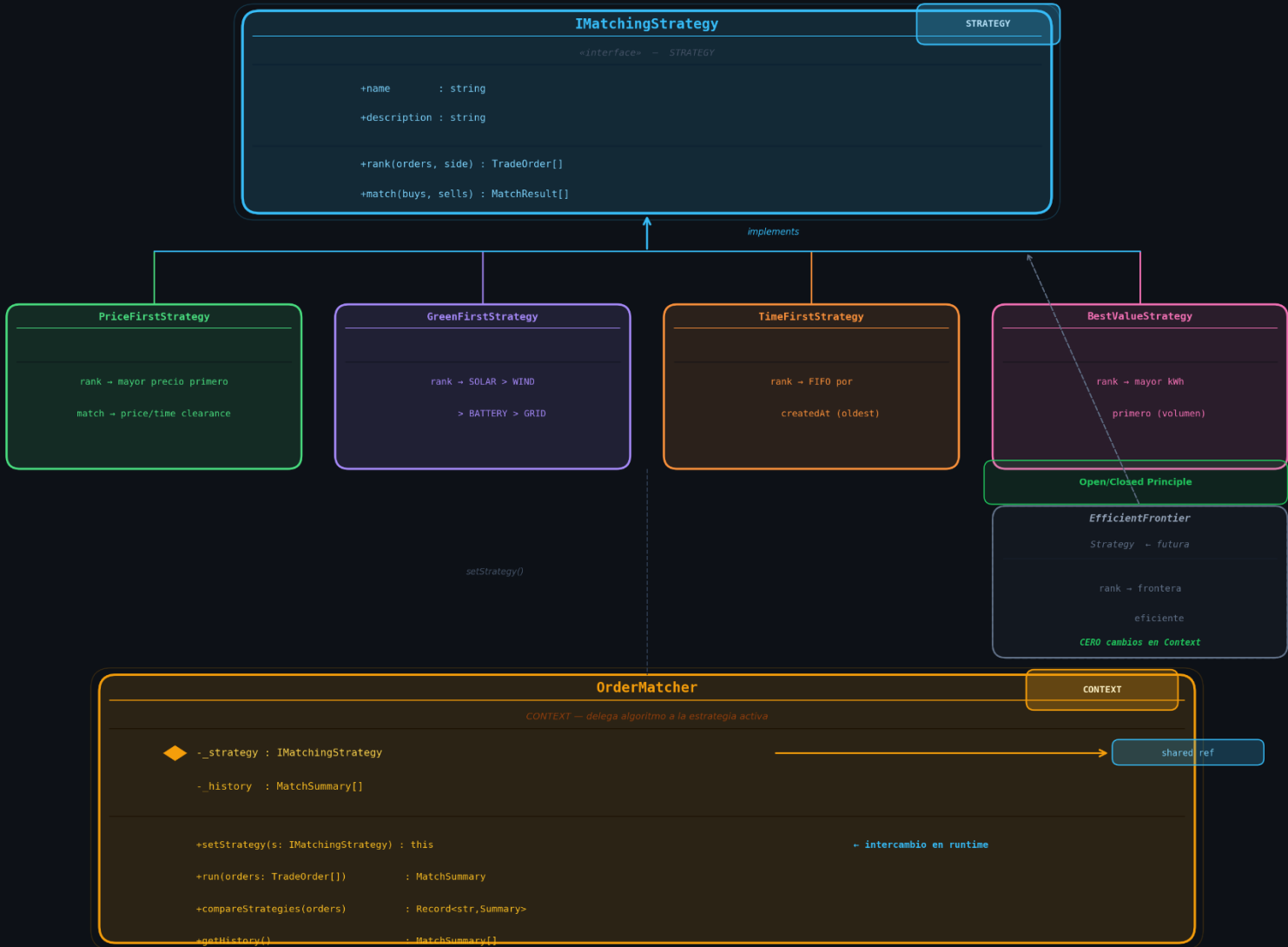
```
interface IMatchingStrategy {
  name: string;
  description: string;
  rank(orders, side): TradeOrder[]; // ordena las órdenes según criterio
  match(buyOrders, sellOrders): MatchResult[]; // empareja compras con ventas
}
```

encapsular cada algoritmo en una clase separada que implementa IMatchingStrategy. El contexto (OrderMatcher) recibe la estrategia en runtime y la delega sin conocer su lógica interna.

Diagrama Uml

PATRON STRATEGY — Order Matching System

El Context (OrderMatcher) delega el algoritmo de matching a la estrategia activa en runtime



USO EN RUNTIME

```
const matcher = new OrderMatcher(new PriceFirstStrategy())

matcher.run(orders) // usa PriceFirst

matcher.setStrategy(new GreenFirstStrategy())

matcher.run(orders) // usa GreenFirst (mismo Context)

matcher.compareStrategies(orders) // prueba todas en paralelo
```

Patrón 2: Command

¿Qué problema resuelve?

Las operaciones sobre dispositivos IoT se llaman directamente

```
// ❌ Sin Command – llamadas directas, sin historial, sin undo:  
await supabase.from("iot_devices").update({ status: "OFFLINE" }).eq("id", "D1");  
// Si fue un error → no hay forma de revertirlo  
// No hay log de qué se hizo ni cuándo  
// No se puede encolar ni programar
```

Con Command, cada operación es un objeto que sabe ejecutarse Y deshacerse

```
// ✅ Con Command – operaciones controladas:  
const cmd = new TurnOffCommand("D1", service);  
await bus.execute(cmd); // ejecuta y guarda en historial  
await bus.undo();       // deshace → el dispositivo vuelve a ONLINE  
await bus.redo();       // rehace  
bus.getHistory();       // ¿quién hizo qué y cuándo?
```

¿Dónde se usa?

```
interface IDeviceCommand {  
  commandType: string;  
  deviceId: string;  
  description: string;  
  execute(): Promise<CommandResult>; // hace la operación  
  undo(): Promise<CommandResult>; // la revierte  
  canUndo(): boolean; // ¿se puede deshacer?  
}
```

encapsula cada operación en un objeto Command que implementa execute() y undo(). El Invoker (DeviceCommandBus) mantiene el historial y controla la ejecución.


```

class DeviceCommandBus {
  private _undoStack: IDeviceCommand[] = [];
  private _redoStack: IDeviceCommand[] = [];
  private _history: CommandRecord[] = [];

  async execute(cmd) {
    const result = await cmd.execute(); // delega al Receiver
    this._history.push({ cmd, result }); // guarda en historial
    if (result.ok) {
      this._undoStack.push(cmd);
      this._redoStack = []; // nuevo cmd invalida redo
    }
  }

  async undo() {
    const cmd = this._undoStack.pop();
    const result = await cmd.undo(); // revierte
    this._redoStack.push(cmd);
    return result;
  }

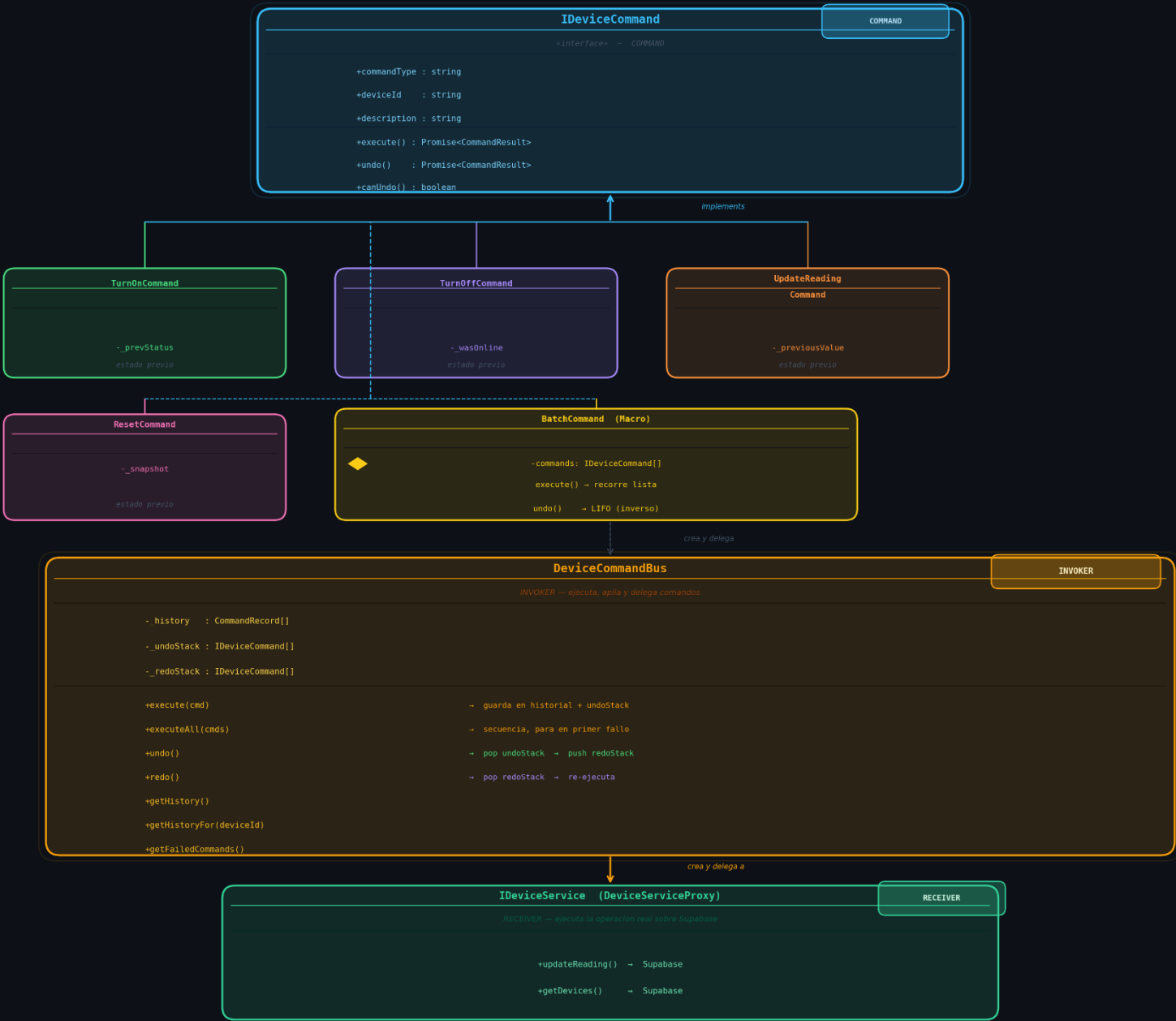
  async redo() {
    const cmd = this._redoStack.pop();
    await this.execute(cmd); // re-ejecuta
  }
}

```

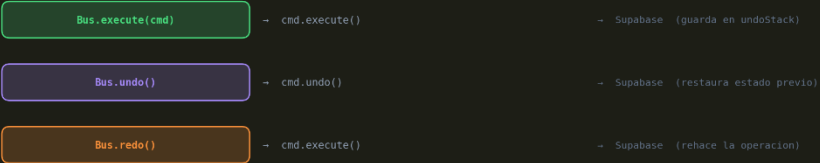
Diagrama Uml

PATRON COMMAND — DeviceCommandBus

Encapsula operaciones como objetos — soporta execute, undo, redo e historial



FLUJO EXECUTE / UNDO / REDO



Patrón 3: Observer

¿Qué problema resuelve?

cuando el precio de mercado cambia, múltiples subsistemas necesitan reaccionar:

- El logger debe registrar el cambio
- El bridge debe enviar una alerta si es un spike
- El matcher debe re-evaluar órdenes pendientes
- La UI debe mostrar una notificación

```
* IMarketObserver: contrato que deben cumplir todos los suscriptores.
* Cada Observer declara qué tipos de eventos le interesan.
*/
export interface IMarketObserver {
  readonly observerId: string;
  readonly observerName: string;
  readonly interestedIn: MarketEventType[]; // filtro declarativo

  onEvent(event: MarketEvent): void | Promise<void>;
}
```

Sin el patrón Observer, el código encargado de actualizar los datos tendría que llamar directamente a cada uno de estos subsistemas. Esto acoplaría fuertemente los componentes, haciendo que agregar un nuevo suscriptor implique modificar la lógica del emisor.

¿Dónde se usa?

El Subject (MarketEventBus) emite eventos sin conocer a sus suscriptores. Cada Observer decide si le interesa el evento y cómo reaccionar. Completamente desacoplado.

```

export class MarketEventBus {
    private static _instance: MarketEventBus | null = null;

    private readonly _observers    = new Map<string, IMarketObserver>();
    private readonly _eventLog:    MarketEvent[] = [];
    private readonly _maxLogSize:  number;
    private _emitCount             = 0;

    constructor(maxLogSize = 1000) { this._maxLogSize = maxLogSize; }

    /** Singleton para uso global */
    static getInstance(): MarketEventBus {
        if (!MarketEventBus._instance)
            MarketEventBus._instance = new MarketEventBus();
        return MarketEventBus._instance;
    }
}

```

Observadores Concretos:

PriceLogObserver: Almacena y promedia en memoria las cotizaciones recibidas.

AlertObserver: Utiliza el Bridge de notificaciones existente en el proyecto para alertar de desconexiones o picos bruscos de tarifa.

OrderMatchingObserver: Evalúa cuándo re-ejecutar el algoritmo de matching al detectar fluctuaciones de precio que superen un umbral configurado (ej. 2%).

UIFeedObserver: Provee un feed FIFO para consumo de la interfaz de usuario.

```

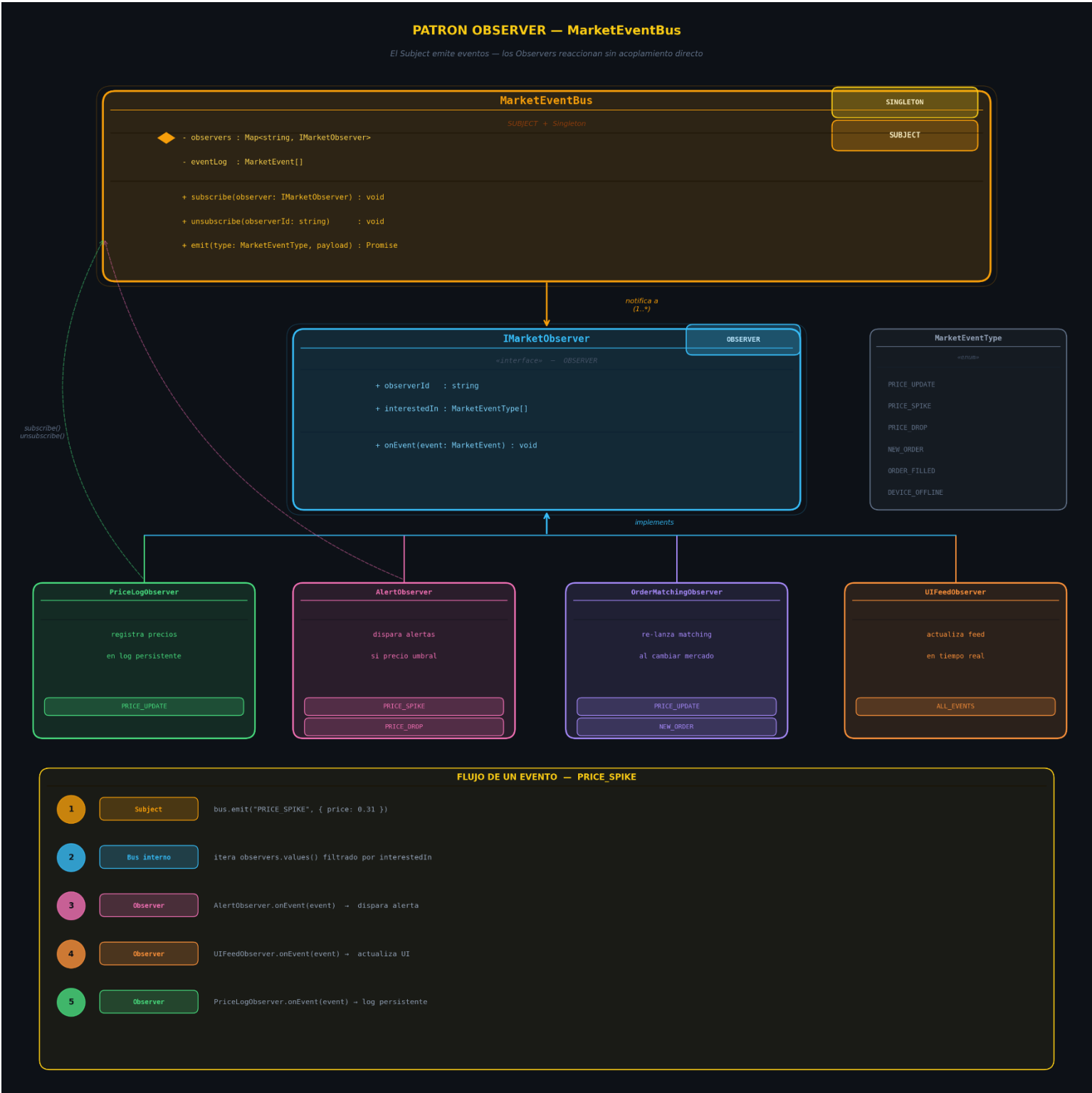
export class UIFeedObserver implements IMarketObserver {
    readonly observerId    = "ui-feed";
    readonly observerName = "Feed de la interfaz";
    readonly interestedIn: MarketEventType[] = [
        "PRICE_UPDATED", "PRICE_SPIKE", "PRICE_DROP",
        "ORDER_MATCHED", "ORDER_EXPIRED", "ORDER_CANCELLED", "ORDER_PUBLISHED",
        "DEVICE_OFFLINE", "DEVICE_ONLINE", "MARKET_SUMMARY_READY",
    ];

    private readonly _feed: MarketEvent[] = [];
    private readonly _maxSize: number;

    constructor(maxSize = 50) { this._maxSize = maxSize; }
}

```

Diagrama Uml



Patrón 4: State

¿Qué problema resuelve?

una orden de trading pasa por varios estados DRAFT → OPEN → MATCHED | EXPIRED | CANCELLED

Sin State: el código usa strings ("OPEN", "MATCHED") y cualquier parte puede asignar cualquier estado sin validación:

```
order.status = "MATCHED"; // ¿pero estaba OPEN? ¿o DRAFT?
```

```
order.status = "CANCELLED"; // ¿puede cancelarse una MATCHED?
```

Con el patrón State, cada estado del ciclo de vida se modela como una clase independiente. Cada clase implementa las operaciones válidas y bloquea con excepciones estructuradas (OrderStateError) cualquier transición ilícita.

¿Dónde se usa?

OrderContext en OrderStateMachine.ts. Envuelve a la orden original, almacena el estado actual y mantiene un historial auditable de transiciones con marcas de tiempo (StateTransition[]).

```
export interface IOrderState {
  readonly statusName: OrderStatus;

  /** DRAFT → OPEN: publica la orden en el mercado */
  publish(ctx: OrderContext): void;

  /** OPEN → MATCHED: empareja la orden con una contraparte */
  match(ctx: OrderContext, meta: MatchMetadata): void;

  /** DRAFT | OPEN → CANCELLED: cancela la orden */
  cancel(ctx: OrderContext, reason: string): void;

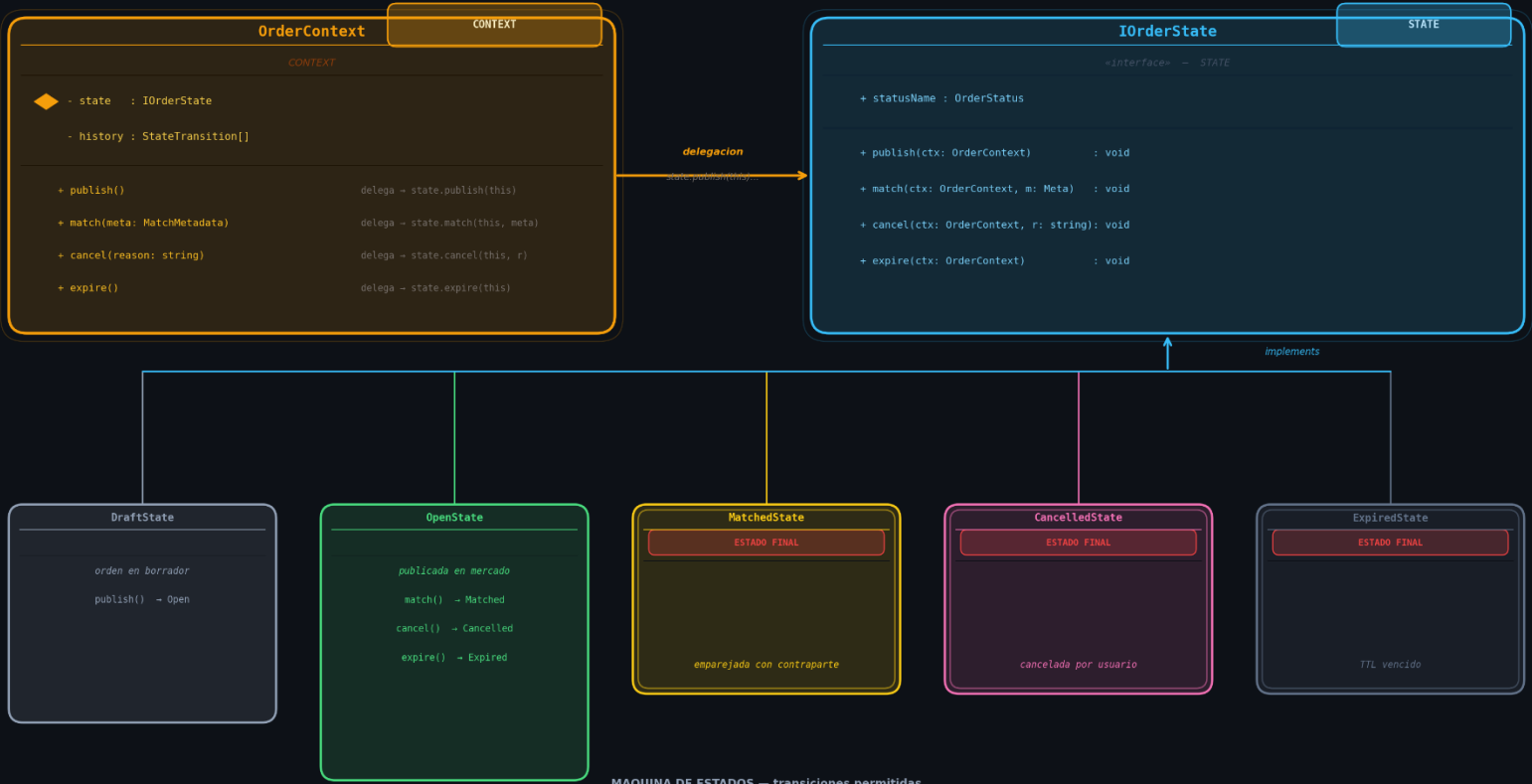
  /** OPEN → EXPIRED: la orden venció sin ejecutarse */
  expire(ctx: OrderContext): void;

  /** Descripción del estado para UI */
  describe(): string;
}
```

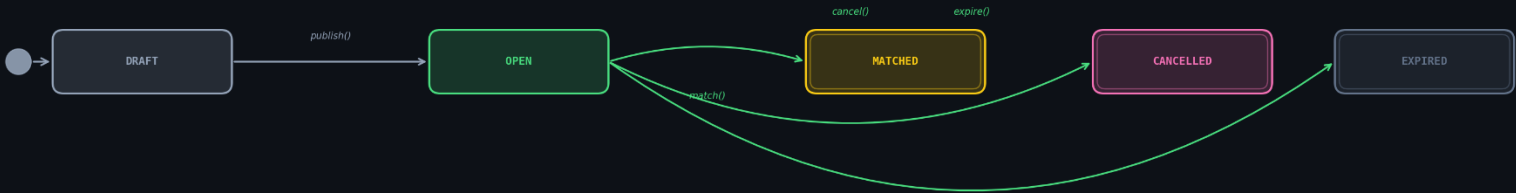
Diagrama Uml

PATRON STATE — Order Lifecycle

OrderContext delega cada accion al estado activo — el estado decide la transicion siguiente



MAQUINA DE ESTADOS — transiciones permitidas



REGLAS DEL PATRON STATE

- Context no conoce los estados concretos — solo habla con IOrderState
- Cada estado decide si la accion es valida y cuál es el estado siguiente
- Estado inválido → lanza error en lugar de hacer nada silenciosamente
- Estados finales (Matched, Cancelled, Expired) ignoran todas las acciones
- `ctx.setState(new OpenState())` ← el estado hijo cambia el contexto

5 resultados del proyecto

1. Dashboard unificado con vista en tiempo real

Se logró una vista centralizada del sistema que muestra órdenes activas, dispositivos IoT, resumen de mercado y plantillas disponibles en una sola pantalla, sin que el componente React conozca la complejidad interna de los subsistemas.

Facade, Composite, Singleton

2. Sistema de creación y publicación de órdenes de trading

Se implementó un flujo completo de creación de órdenes con API fluida (Builder), recetas rápidas predefinidas (Director) y clonación de plantillas existentes (Prototype), permitiendo crear y publicar órdenes en un solo paso desde la UI.

Builder, Prototype, Facade

3. Mercado energético con precios de múltiples proveedores externos

Se integraron tres proveedores de precios incompatibles (OMIE en €/MWh, ENTSO-E y Octopus) bajo una interfaz estándar única, con caché TTL, logging, alertas de spike y reintentos automáticos apilados como capas independientes.

Adapter, Decorator, Singleton

4. Gestión de dispositivos IoT con control de acceso por roles

Se implementó la creación de dispositivos IoT usando Factory Method (protocolos MQTT, REST, Simulated), Abstract Factory (familias Solar, Eólica, Batería, Red) y Proxy con control de roles (VIEWER / OPERATOR / ADMIN), caché de 30 s y auditoría de cada operación.

Factory Method, Abstract Factory, Proxy, Flyweight

5. Sistema de notificaciones multi-canal con alertas inteligentes

Se entregó un sistema que dispara alertas de precio, estado de órdenes y dispositivos offline a través de cuatro canales (consola, in-app, email, webhook), reduciendo de 12 clases posibles a 7 mediante Bridge, y usando el Decorator como disparador automático de alertas al superar umbrales de precio.

Bridge, Decorator